CalCOFI.io

Documentation

Benjamin D. Best Erin Satterthwaite

2026-09-03

Table of contents

1 Process

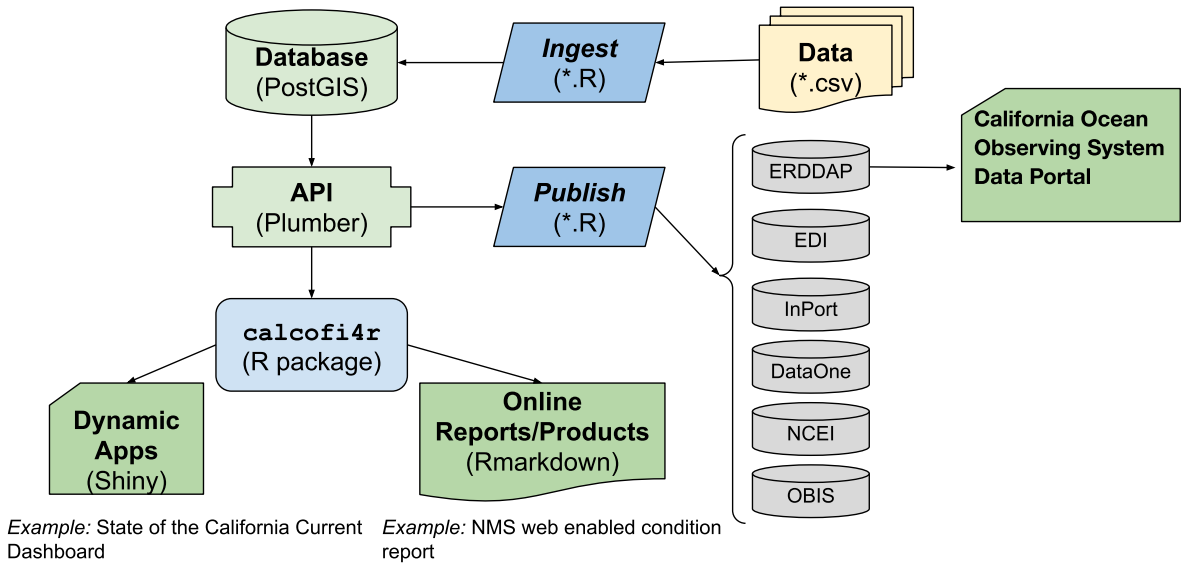


Figure 1.1: CalCOFI data workflow.

The original raw **data**, most often in tabular format [e.g., comma-separated value (*.csv)], gets **ingest**ed into the **database** by R scripts that use functions and lookup data tables in the R package `calcofi4r` where functions are organized into *Read*, *Analyze* and *Visualize* concepts. The application programming interface (**API**) provides a program-language-agnostic public interface for rendering subsets of data and custom visualizations given a set of documented input parameters for feeding interactive applications (**Apps**) using Shiny (or any other web application framework) and **reports** using Rmarkdown (or any other report templating framework). Finally, R scripts will **publish** metadata (as Ecological Metadata Language) and data packages (e.g., in Darwin format) for discovery on a variety of data ***portals*** oriented around slicing the tabular or gridded data (ERDDAP), biogeographic analysis (OBIS), long-term archive (DataOne, NCEI) or metadata discovery (InPort). The **database** will be spatially enabled by PostGIS for summarizing any and all data by ***Areas of Interest*** (AoIs), whether pre-defined (e.g., sanctuaries, MPAs, counties, etc.) or arbitrary new areas. (Figure 1.1)

2 Reports

2.1 Sanctuaries

- Channel Islands WebCR
  web-enabled Condition Report
  - Forage Fish
    example of using calcofi4r functions that pull from the API
- UCSB Student Capstone

3 Applications

- CalCOFI Schema
  per-release schema browser: ERD, tables, columns (with units + descriptions), datasets, measurement-type registry — sourced from the release `metadata.json` sidecar on GCS
- CalCOFI Query
  browser-only DuckDB-WASM playground against the public release Parquet on GCS
- CalCOFI Oceanography
  oceanographic summarization by arbitrary area of interest and sampling period
- UCSB Student Capstone

4 Maps

4.1 Overview

The CalCOFI interactive app renders **hexagonal heatmaps** of species occurrence and oceanographic variables — sardine larvae densities, sea-surface temperature, salinity, chlorophyll — across the entire CalCOFI sampling area, at any zoom level, in seconds. The engine that makes this possible is **H3T**, a tile service that converts a database query into a stream of hexagon tiles on demand.

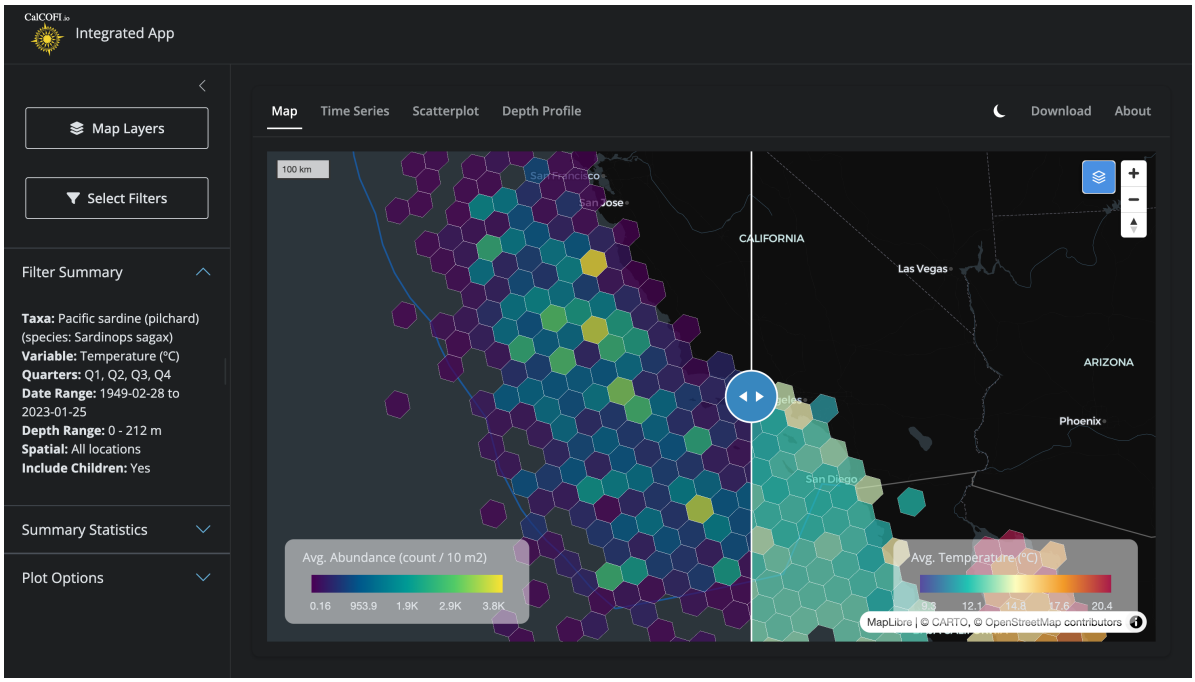


Figure 4.1: Side-by-side comparison in the CalCOFI Integrated App: hexagons summarizing **biology** (Pacific sardine larvae abundance, left) and **environment** (sea-surface temperature, right) over the same area, era, and quarters, with a swipeable divider for direct visual comparison. Both layers are powered by the H3T tile service.

This chapter explains how H3T works, visually, for both the curious oceanographer and the developer who wants to extend it. Throughout, the guiding principle is:

DuckDB is the single source of truth. H3T tiles are a performance layer built on top of it — never an authoritative copy of the data.

4.2 Why hexagons?

H3 is Uber's hexagonal hierarchical geospatial index. Each cell is a hexagon, and every cell at resolution N nests neatly inside one parent cell at resolution $N - 1$. Hexagons have three useful properties for ecological data:

- **Equal area at a given resolution**, so densities are directly comparable across the map.
- **Every neighbor is the same distance away** (unlike a square grid, which has both edge and corner neighbors).
- **No latitude distortion** — a res-5 hex off Point Conception covers the same area as a res-5 hex off Cabo San Lucas.

Compared to a raster of pixels, hexagons aggregate point observations (a net tow, a CTD cast, a larvae count) without imposing an arbitrary north–south orientation, and the parent–child nesting lets the same dataset render coarsely when you're zoomed out and finely when you're zoomed in.

4.3 The problem: aggregating millions of points at every zoom

Before H3T, the db-viz-hex pre-computed every hex aggregation across **all 10 H3 resolutions** for every species and environmental variable at startup. The Shiny app then shipped the relevant resolution to the browser for each zoom change. This gave the app two problems:

1. **Slow startup.** Building those aggregations against the full bottle and ichthyo tables took >10 s every time the app launched.
2. **Heavy memory.** A multi-resolution `sf` grid for one variable is hundreds of megabytes; for dozens of variables it crowded out everything else on the Shiny server.

This is the same class of problem that the MarineSensitivity stack solved for rasters with a TiTiler factory + Varnish cache. H3T is the equivalent move for **hexagons**.

4.4 The solution: query-as-URL hexagonal tiles

The architecture rests on three ideas, each covered below: the H3 resolution that's served depends on the zoom level; the entire tile request lifecycle is fronted by a cache; and a `release` parameter handles cache invalidation automatically when new data drops.

4.4.1 Resolution follows zoom

When the user zooms out to see the whole California Current, they don't need 7-meter hexagons — a 22 km hex is finer than the underlying sampling grid anyway. When they zoom in to a single CalCOFI line, they want as much detail as the data supports. H3T maps web-map zoom levels to H3 resolutions on the server side: zoom 0–1 → res 1 (~1106 km hexes), zoom 5–7 → res 5 (~22 km hexes), zoom 11+ → res 10 (~7.5 m hexes), and so on (Figure 4.2).

The trick is in the SQL: the db-viz-hex sends a **template** with a literal placeholder `hex_h3res{{res}}` (see `build_sp_sql()` in db-viz-hex/app/functions_h3t.R), and the API substitutes the right resolution for each tile before executing. One query string, ten zoom levels, one shared cache namespace.

4.4.2 Request lifecycle

When the user pans or zooms, MapLibre asks for tiles. Each tile URL looks like this:

```
h3tiles://h3t.calcofi.io/h3t/{z}/{x}/{y}.h3t?q=<base64-SQL>&release=v2026.04.08
```

The clever bit is the `q` parameter: it's the entire `SELECT` statement, base64-encoded with URL-safe characters (RFC 4648 §5). That means **the URL itself describes the data to render**, so two users asking for "average temperature for sardines, all quarters, 1949–2024" hit the exact same cache entry — no session state, no server-side query registry.

What happens next is shown in Figure 4.3. Caddy terminates TLS and forwards to Varnish, which checks its cache. On a hit, the user gets back a chunk of h3j JSON in well under 100 ms. On a miss, Varnish forwards to the Plumber API, which:

1. **Decodes** the base64 SQL.
2. **Validates** it with sqlglot: single statement, must be `SELECT` (or `WITH … SELECT`), must project exactly `cell_id, value, [n]`, no `read_csv`/`attach`/`load`, no shell calls, no schema beyond the published tables.
3. **Substitutes** `{res}` from the zoom level and wraps the inner query in a viewport bounding-box filter.
4. **Executes** against a read-only DuckDB connection with a 3-second timeout and a 50,000-row cap.
5. **Formats** the result as h3j JSON: `{"cells": [{"h3id": "8a1941a29a7ffff", "value": 12.37, "n": 42}, …]}`.

The validation, read-only mode, timeout, and row cap are stacked guardrails: the public URL accepts arbitrary SQL because **none of the layers will let it do harm**. Even if a determined actor slipped a malformed statement past sqlglot, the DuckDB connection has no write permission, no extension load, and a hard execution budget.

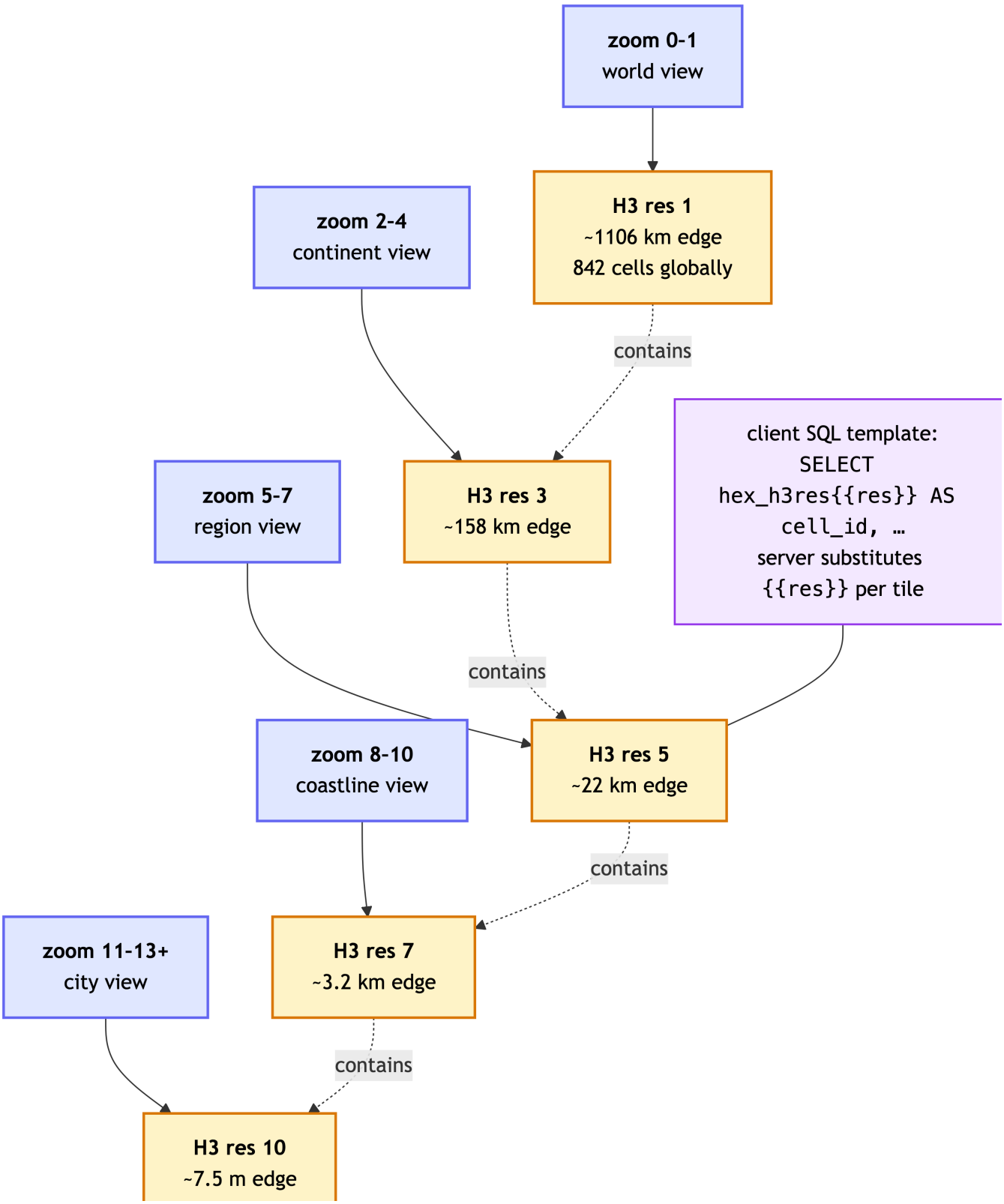


Figure 4.2: Web-map zoom level determines which H3 resolution the server returns. The same SQL template — `hex_h3res{{res}}` — adapts to each tile by server-side substitution. Hexagons at finer resolutions nest inside their coarser parents, so the data hierarchy and the visual hierarchy stay aligned.

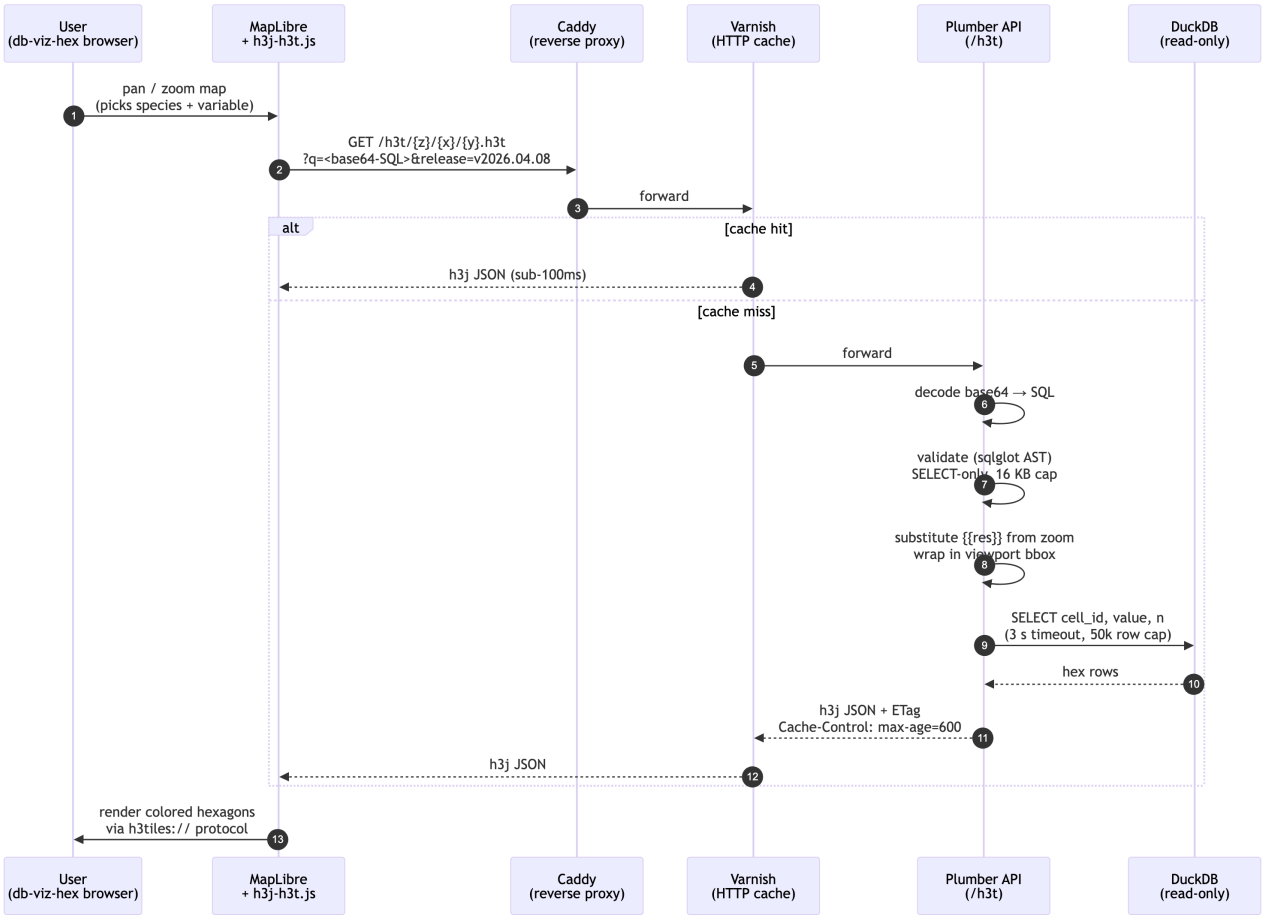


Figure 4.3: Tile request lifecycle. The browser asks for a tile by URL; Varnish caches identical URLs; on a miss, the Plumber API decodes and validates the SQL, runs it against a read-only DuckDB, and returns h3j-formatted hexagons. Validation + read-only DB + timeouts mean the public endpoint can accept arbitrary SQL safely.

4.4.3 Cache invalidation by release

Caches are easy to fill but notoriously hard to clear. H3T sidesteps the hard part entirely with a single trick: the `release` query parameter (Figure 4.4).

When CalCOFI publishes a new database release (e.g., `v2026.04.08`, see Database), a symlink at the API server is swapped to point at the new file. The next time the db-viz-hex loads, it asks `/h3t/meta` for the current release version, gets back `v2026.04.08`, and passes that into every subsequent tile URL. Because the URL is now different, Varnish **automatically cache-misses** — and the new tiles are generated against the fresh data on first request. Old tiles linger harmlessly in cache until evicted by LRU; they'd still serve correct data for their old release if anyone asked.

No purge command, no cache key surgery. Just: change one parameter, get a clean cache layer for free.

4.5 A worked example: sardine + temperature

To make this concrete, here is the full life of one click in the db-viz-hex:

1. **User picks** *Pacific sardine* (`Sardinops sagax`) and *temperature* in the sidebar, with date range 1949–2024 and all four quarters.
2. `build_sp_sql()` assembles a SQL template:

```
SELECT hex_h3res{{res}} AS cell_id,
       AVG(std_tally)   AS value,
       COUNT(*)         AS n
  FROM bio_obs
 WHERE scientific_name = 'Sardinops sagax'
   AND quarter IN (1, 2, 3, 4)
   AND time_start BETWEEN '1949-01-01' AND '2024-12-31'
 GROUP BY 1
```

3. `h3t_b64()` base64-encodes the SQL → `q=U0VMRUNUIGhleF9oM3Jlc3t7cmVzfX0gQVMgY2VsbF9pZCwKICAgIC`
4. `fetch_h3t_stats()` calls `/h3t/stats?q=…&release=v2026.04.08` once to get `{min, max, p02, p98, n}`. The 2nd–98th percentiles drive a robust color scale that ignores outliers.
5. **MapLibre** requests tiles as the user pans:

   `h3tiles://h3t.calcofi.io/h3t/4/3/6.h3t?q=U0VMRUNU…&release=v2026.04.08`

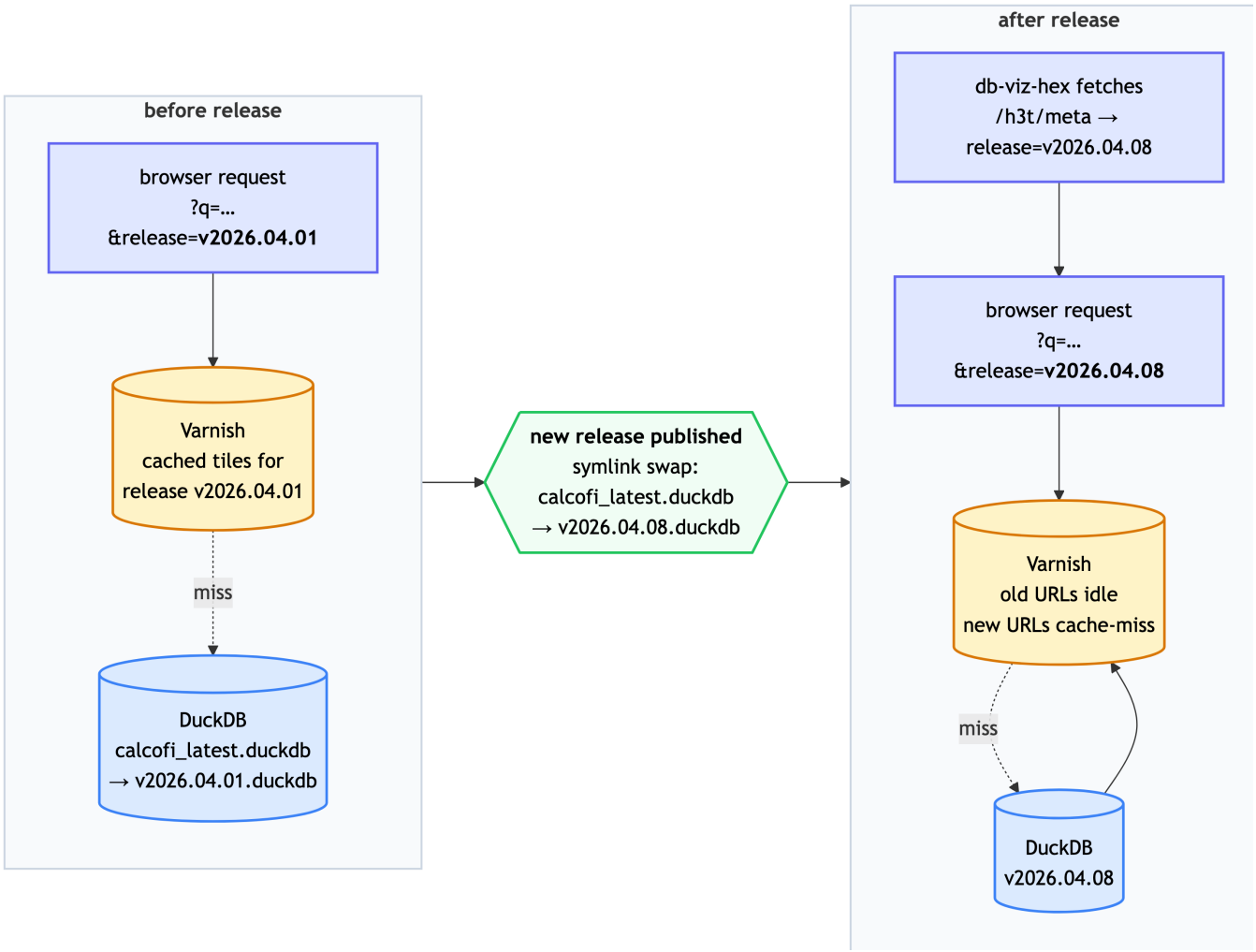


Figure 4.4: Cache invalidation by release. The `release` query parameter is part of every tile URL. When a new database release is published, the db-viz-hex picks up the new version from `/h3t/meta`, all subsequent tile URLs change, and Varnish refills naturally — no manual purge.

6. **Plumber** decodes, substitutes `{res}` (zoom 4 → res 3), wraps in a bounding-box filter for tile (4, 3, 6), runs the query, and returns:


```
{
  "cells": [
    { "h3id": "83390cfffffffff", "value": 12.37, "n": 42 },
    { "h3id": "83390dfffffffff", "value": 14.02, "n": 38 },
    { "h3id": "83390efffffffff", "value": 11.85, "n": 51 }
  ]
}
```


7. **MapLibre** paints each hex by interpolating the stats palette between p02 and p98. The user sees a smooth choropleth update as new tiles arrive.

The first time this query runs anywhere, it takes ~200–800 ms (Varnish miss, DuckDB execution). Every subsequent user requesting "sardines, all quarters, 1949–2024" hits Varnish and renders in under 100 ms.

4.6 How the data gets there

The H3T service is the *last* link in the CalCOFI data pipeline, not the first:

- Source data (Google Drive, NCEI, ERDDAP, SWFSC) is ingested through Quarto notebooks in `CalCOFI/workflows` into a working DuckDB database.
- A `release_database.qmd` step validates, freezes, and publishes a versioned DuckDB file to the API server.
- The H3T API mounts that file read-only and serves tiles from it.

For the schema details, see Database. For the publish path, see Portals.

4.7 Repositories and code

Repository	Role
CalCOFI/api-h3t	Plumber API, sqlglot validator, DuckDB driver, deployment configs
CalCOFI/db-viz-hex	Shiny consumer; SQL builders and map widgets in `app/functions_h3t.R`
bbest/mapgl @ feat/add-h3t-source	`add_h3t_source()` extension to the mapgl R package

Repository	Role
bbest/h3j-h3t @ fix/maplibre-v3-compat	fix h3t protocol for MapLibre v3+/v4 and multiple sources on one page in INSPIDE/h3j-h3t

For the full technical contract — every endpoint, every validation rule, every deployment knob — see the `api-h3t` README and `deploy.md`.

5 Database

5.1 Database naming conventions

> There are only two hard things in Computer Science: cache invalidation and naming things. – Phil Karlton (Netscape architect)

We're circling the wagons to come up with the best conventions for naming. Here are some ideas:

- Learn SQL: Naming Conventions
- Best Practices for Database Naming Conventions - Drygast.NET

5.1.1 Name tables

- Table names are **singular** and use all lower case.
  - Example: `cruise`, `site`, `species`, `lookup` (not `cruises`, `sites`, `lookups`)

5.1.2 Name columns

- To name columns, use **snake-case** (i.e., lower-case with underscores) so as to prevent the need to quote SQL statements. (TIP: Use `janitor::clean_names()` to convert a table.)
- Unique **identifiers** are suffixed with:
  - `*_id` for unique integer keys;
  - `*_uuid` for universally unique identifiers as defined by RFC 4122 and stored in Postgres as UUID Type.
  - `*_key` for unique string keys;
  - `*_seq` for auto-incrementing sequence integer keys.
- Suffix with **units** where applicable (e.g., `*_m` for meters, `*_km` for kilometers, `degc` for degrees Celsius). See units vignette.

- Set geometry column to **geom** (used by PostGIS spatial extension). If the table has multiple geometry columns, use `geom` for the default geometry column and `geom_{type}` for additional geometry columns (e.g., `geom_point`, `geom_line`, `geom_polygon`).

5.1.3 Primary key conventions

Prefer natural keys (meaningful domain identifiers) over surrogate keys where stable:

- `cruise_key` = 'YYMMKK' (e.g., `2401NH` = January 2024, New Horizon)
- `ship_key` = 2-letter ship code (e.g., `NH`)
- `tow_type_key` = tow type code (e.g., `CB`)

Sequential integer keys should have explicit sort order documented for reproducibility:

- `site_id` sorted by `cruise_key, orderocc`
- `tow_id` sorted by `site_id, time_start`
- `net_id` sorted by `tow_id, side`
- `ichthyo_id` sorted by `net_id, species_id, life_stage, measurement_type, measurement_value`

Avoid UUIDs in output tables: Use `_source_uuid` in Working DuckLake for provenance tracking only (stripped in frozen releases).

5.2 Use Unicode for text

The default character encoding for Postgresql is unicode (`UTF8`), which allows for international characters, accents and special characters. Improper encoding can royally mess up basic text.

Logging into the server, we can see this with the following command:

```
docker exec -it postgis psql -l
```

```
                                  List of databases
        Name         | Owner | Encoding |  Collate   |   Ctype    | Access privileges
---------------------+-------+----------+------------+------------+-------------------
 gis                 | admin | UTF8     | en_US.utf8 | en_US.utf8 | =Tc/admin         +
                     |       |          |            |            | admin=CTc/admin   +
                     |       |          |            |            | ro_user=c/admin
 lter_core_metabase  | admin | UTF8     | en_US.utf8 | en_US.utf8 | =Tc/admin         +
                     |       |          |            |            | admin=CTc/admin   +
                     |       |          |            |            | rw_user=c/admin
 postgres            | admin | UTF8     | en_US.utf8 | en_US.utf8 |
```

```
 template0          | admin | UTF8     | en_US.utf8 | en_US.utf8 | =c/admin         +
                    |       |          |            |            | admin=CTc/admin
 template1          | admin | UTF8     | en_US.utf8 | en_US.utf8 | =c/admin         +
                    |       |          |            |            | admin=CTc/admin
 template_postgis   | admin | UTF8     | en_US.utf8 | en_US.utf8 |
(6 rows)
```

Use Unicode (`utf-8` in Python or `UTF8` in Postgresql) encoding for all database text values to support international characters and documentation (i.e., tabs, etc for markdown conversion).

- In **Python**, use `pandas` to read (`read_csv()`) and write (`to_csv()`) with UTF-8 encoding (i.e., `encoding='utf-8'`).:

```python
import pandas as pd
from sqlalchemy import create_engine
engine = create_engine('postgresql://user:password@localhost:5432/dbname')

# read from a csv file
df = pd.read_csv('file.csv', encoding='utf-8')

# write to PostgreSQL
df.to_sql('table_name', engine, if_exists='replace', index=False, method='multi', chunks

# read from PostgreSQL
df = pd.read_sql('SELECT * FROM table_name', engine, encoding='utf-8')

# write to a csv file with UTF-8 encoding
df.to_csv('file.csv', index=False, encoding='utf-8')
```

- In **R**, use `readr` to read (`read_csv()`) and write (`write_excel_csv()`) to force UTF-8 encoding.

```r
library(readr)
library(DBI)
library(RPostgres)

# connect to PostgreSQL
con <- dbConnect(RPostgres::Postgres(), dbname = "dbname", host = "localhost", port = 54

# read from a csv file
df <- read_csv('file.csv', locale = locale(encoding = 'UTF-8'))  # explicit
df <- read_csv('file.csv')                                       # implicit
```

```r
# write to PostgreSQL
dbWriteTable(con, 'table_name', df, overwrite = TRUE)

# read from PostgreSQL
df <- dbReadTable(con, 'table_name')

# write to a csv file with UTF-8 encoding
write_excel_csv(df, 'file.csv', locale = locale(encoding = 'UTF-8'))  # explicit
write_excel_csv(df, 'file.csv')                                       # implicit
```

5.3 Integrated database ingestion strategy

5.3.1 Overview

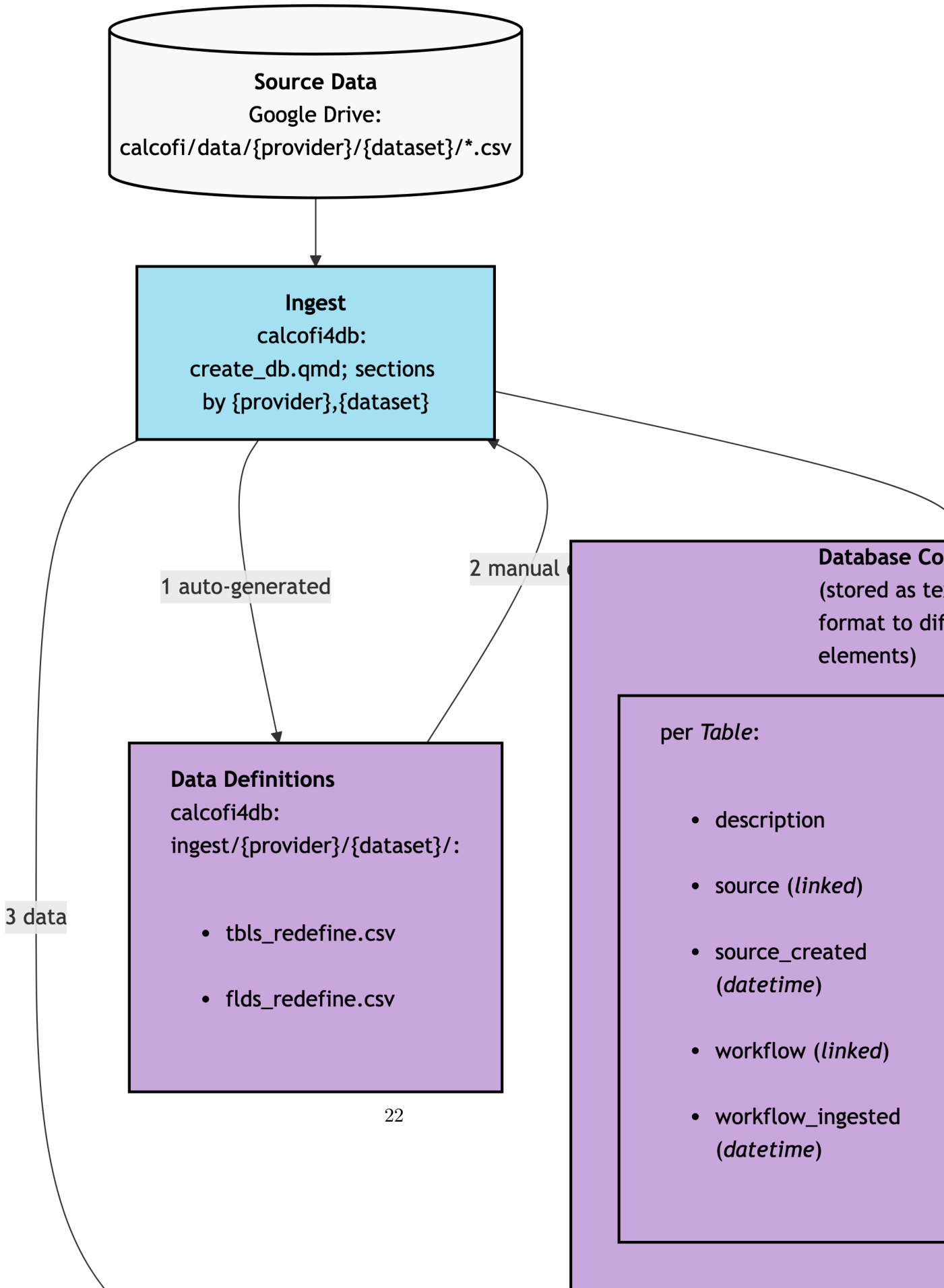
The integrated database is built by the `CalCOFI/workflows` pipeline: one `ingest_{provider}_{dataset}.qmd` notebook per dataset (orchestrated by `targets`, engine functions in `calcofi4db`) writes Parquet shards plus JSON sidecars; `release_database.qmd` assembles them in an in-memory DuckDB, validates, and freezes a versioned, immutable **release** on public Google Cloud Storage (next section). Consumers — the apps, `calcofi4r::cc_get_db()`, the schema and query sites — read the release as Parquet over HTTPS with no credentials.

Alongside the releases, a **PostgreSQL** database (`calcofi`, PostgreSQL 18 + PostGIS 3.6 on the CalCOFI server) is the *working*, multi-user store for the CTD team's QA/QC: originals loaded verbatim and immutable, a flag/proposal ledger beside them, derived products on top. It is private and reached over SSH — see Server Access. DuckDB's `postgres` extension joins the two worlds in one query (`calcofi4r::cc_pg_attach()`). (An earlier design ran the whole database in PostgreSQL with `dev`/`prod` schemas and a single `create_db.qmd`; that is superseded and only the legacy `gis` database remains, behind tile.calcofi.io.)

5.3.2 Release versioning

Each rendered `release_database.qmd` writes a frozen, immutable release to `gs://calcofi-db/ducklake/releases/{version}/` where `version` is `vYYYY.MM.DD`. The release directory holds the **sidecars** — the complete record of what the release is — and the `catalog.json` among them is the data-access contract: consumers resolve every table through it (Data Access) rather than building a path.

- `catalog.json` — structural manifest: `version`, `release_date`, `total_rows` / `total_size`, and per table `name`, `rows`, `partitioned`, `supplemental`. From the v2026.09 releases it is **content-addressed** (`layout: "canonical"`, plus the


Source Data
Google Drive:
calcofi/data/{provider}/{dataset}/*.csv
Ingest
calcofi4db:
create_db.qmd; sections
by {provider},{dataset}
1 auto-generated
2 manual
Data Definitions
calcofi4db:
ingest/{provider}/{dataset}/:
tbls_redefine.csv
flds_redefine.csv
3 data
Database Co
(stored as te
format to dif
elements)
per Table:
description
source (linked)
source_created
(datetime)
workflow (linked)
workflow_ingested
(datetime)

`writer` settings — compression, row-group size — that make the bytes reproducible): each table carries `content_hash` (a whole-table signature), `compat_path` (its legacy per-release path) and `objects[]` — one object for a single-file table, one per partition for a partitioned one — each with a bucket-relative `path` (`ducklake/tables/{table}/{content_hash}/{table}.parquet`, or `ducklake/tables/{table}/{col}=`
`bytes`, `sha256`, `content_hash`, `since` (the first version that shipped this exact object), its own `compat_path` and, for a partition, `partition_by` / `partition_value`. A partitioned table may also publish a whole-table **twin** — the one object without `partition_by` (`obs` does) — for https-only readers; it is read *instead of* the partitions, never alongside them. Objects are immutable and shared between versions, so `since` is the per-table "what changed"; `calcofi4r::cc_release_sources()` and `calcofi4py.release_sources()` are the resolvers (the twin comes back as `single_file`, excluded from `urls`).

- `relationships.json` — primary keys and foreign keys
- `metadata.json` — table and column descriptions, units, dataset block, measurement-type registry
- `RELEASE_NOTES.md` — this version's section of `RELEASES.md` plus a generated appendix (tables and rows from `catalog.json`, datasets, the consumer-contract result, package versions)
- `parquet/` — the legacy per-release copy of the tables (single `.parquet` or hive-partitioned directories). Kept only for the promoted version, its predecessor and the **consolidated** versions; elsewhere it is removed by archive thinning (`metadata/release_policy.yml` in `CalCOFI/workflows`, applied by `scripts/thin_releases.R`), while the sidecars stay.

Bucket-level, under `releases/`: `latest.txt` names the promoted version (promoted only after `test_release.qmd`'s consumer-contract suite passes); `versions.json` lists every release — `version`, `release_date`, `tables`, `total_rows`, `size_mb` — with `consolidated: true` on the milestone versions (`v2026.04.08`, `v2026.05.14`, `v2026.06.26`, `v2026.07.17`, `v2026.08.14`, `v2026.08.25`) and `retired: {retired_utc, to, reason}` on a thinned one, `to` being the consolidated version to use instead; and `RELEASES.md` is the running changelog across versions, newest first, with `# Unreleased` collecting changes until the next cut.

5.3.3 Metadata and documentation

Sources of truth for table and column descriptions live in this repo (not in the database), so they are reviewable via PRs and travel with the code that produced them:

- `metadata/{provider}/{dataset}/tbls_redefine.csv` — `tbl_new`, `tbl_description`
- `metadata/{provider}/{dataset}/flds_redefine.csv` — `tbl_new`, `fld_new`, `fld_description`, `units`
- `metadata/{provider}/{dataset}/metadata_derived.csv` — optional mark-down/units overlay for derived tables and columns

- `metadata/release_tables.csv` and `metadata/release_columns.csv` — for tables built inside `release_database.qmd` itself (`cruise_summary`, `_spatial`, `_spatial_attr`, …)
- `metadata/measurement_type.csv` — canonical measurement types with units
- `metadata/dataset.csv` — dataset-level descriptions, citations, links

These flow through two stages:

1. **Per-ingest**: `calcofi4db::build_metadata_json()` (called inside `finalize_ingest()`) writes `data/parquet/{provider}_{dataset}/metadata.json` (schema version 1.0) alongside the parquet outputs.
2. **Per-release**: `calcofi4db::merge_metadata_json()` (called inside `release_database.qmd`) merges every per-ingest sidecar plus the release-only registries and writes `gs://calcofi-db/ducklake/releases/{version}/metadata.json` (schema version 1.1) with this shape:

```
{
  "schema_version": "1.1",
  "release_version": "v2026.05.14",
  "release_date":    "2026-05-14",
  "datasets":          { "calcofi_bottle": { ... } },
  "tables":            { "bottle":         { "name_long": "Bottle",
                                             "description_md": "...",
                                             "provider": "calcofi",
                                             "dataset":  "bottle" } },
  "columns":           { "bottle.depth_m": { "name_long": "Depth M",
                                             "units":          "meters",
                                             "description_md": "Bottle depth" } },
  "measurement_types": { "temperature":    { "description": "...",
                                             "units":       "degC",
                                             "is_canonical": true } }
}
```

Markdown is supported anywhere in `description_md`; consumers should render it through a markdown parser when displaying.

5.3.4 Consuming descriptions

The fastest way to browse the schema is the **CalCOFI Schema explorer** — per-release ERD, sortable table/column lists with units, dataset provenance, and the measurement-type registry, all sourced from the release `metadata.json` sidecar on GCS. Deep-link to a specific release with `#erd?v=v2026.05.19` (or `#tables`, `#columns`, `#measurements`).

From R, fetch the descriptions either column-by-column or as a full interactive catalog. Both calls hit the same release `metadata.json` sidecar:

```r
library(calcofi4r)

# schema + descriptions + units for a single table
tbl <- cc_describe_table("bottle")
attr(tbl, "description_md")    # table-level description

# interactive datatable of every table and column in the release
cc_db_catalog()
cc_db_catalog(tables = c("bottle", "ichthyo"))
cc_db_catalog(version = "v2026.05.14")
```

When writing a new ingest, populate `tbls_redefine.csv` and `flds_redefine.csv` (especially `fld_description` and `units`) before running the ingest notebook — `build_metadata_json()` reads them directly. The Claude Code `/generate-metadata` and `/validate-ingest` skills enforce this.

5.3.5 Publishing to portals

Workflows in `workflows/publish_*.qmd` push selected tables to external portals (ERDDAP, EDI, OBIS, NCEI). They read the release `metadata.json` to assemble Ecological Metadata Language (EML) and ERDDAP datasets.xml entries, so descriptions stay in sync across portals without re-keying.

5.4 Relationships between tables

- See calcofi/workflows: **clean_db**

- `TODO:` add calcofi/apps: db to show latest tables, columns and relationsips

5.5 Spatial Tips

- Use `ST_Subdivide()` when running spatial joins on large polygons.

6 Data Access

CalCOFI database releases are published as **Parquet** files on a public Google Cloud Storage (GCS) bucket. You can query them directly with DuckDB — from R, Python, the DuckDB CLI, or any DuckDB client — with **no credentials, no API server, and no full download**. DuckDB reads only the columns and row groups a query actually touches, straight over HTTPS.

This page covers querying the release Parquet directly. For convenience-wrapped biological environmental matching, see Matching Helpers.

The releases are read-only snapshots. The CTD team's *working* database — multi-user PostgreSQL on the CalCOFI server, private, reached over SSH — is described in Server Access, including how to join it with the releases from DuckDB.

6.1 Where the data lives

A release is a versioned folder of **sidecars** — the record of what the release is — and its tables are Parquet **objects** that the release's `catalog.json` points at:

```
gs://calcofi-db/ducklake/
  releases/
      latest.txt              # the promoted version, e.g. v2026.08.25
      versions.json           # every version: date, tables, rows; consolidated / retired
      RELEASES.md             # what changed between versions and why, newest first
      {version}/
          catalog.json        # table list, rows, partitioned/supplemental flags,
                               #   and objects[] - where each table's bytes live
          relationships.json  # primary keys + foreign keys
          metadata.json       # table/column descriptions, units, datasets, measurement type
          RELEASE_NOTES.md    # this version's RELEASES.md section + a generated appendix
          parquet/            # legacy per-release copy - promoted + consolidated versions o
              {table}.parquet
              {table}/dataset_key=.../*.parquet
  tables/                      # content-addressed objects, from the v2026.09 releases
      {table}/{content_hash}/{table}.parquet
```

`{table}/{col}={value}/{content_hash}/data_0.parquet`

- Public HTTPS root of the bucket: `https://storage.googleapis.com/calcofi-db/` — every `path` in a catalog is relative to it. `https://storage.calcofi.io/calcofi-db/` serves the same bucket and redirects a legacy `releases/{version}/parquet/…` URL to the canonical object where one still exists.
- The promoted version is at `releases/latest.txt`; the table list — and where each table's bytes live — at `releases/{version}/catalog.json`.
- **Resolve a table through the catalog; never build a `parquet/` path by hand.** From the v2026.09 releases each table entry in `catalog.json` carries `objects[]` — one object for a single-file table, one per partition for a partitioned one — with a bucket-relative `path`, `bytes`, `sha256`, `content_hash`, `since` (the first version that shipped that exact object) and its own `compat_path` (the legacy per-release path of that one file). Objects are immutable and shared between versions: a release that changed three tables shares every other object with the release before it, so `since` is the per-table "what changed". The legacy `releases/{version}/parquet/{table}.parquet` copy remains only for the promoted version and the consolidated ones (see Versions). The packages' resolvers — `calcofi4r::cc_release_sources()`, `calcofi4py.release_sources()` — turn a catalog entry into URLs for any version, including those before v2026.09.
- Most tables are **single-file**. `obs` and the supplemental `obs_ctd_full` / `obs_mets_full` are **hive-partitioned** by `dataset_key` (`catalog.json` flags these with `"partitioned": true`); a partition object's `path` carries its `dataset_key=…` segment, so `hive_partitioning = true` recovers the column. Supplemental tables (`"supplemental": true`) are hosted and cataloged but excluded from `cc_get_db()` by default — `obs_ctd_full` alone is ~270 M rows.
- To **see what's in each table** before writing a query, open the **CalCOFI Schema explorer** — ERD, sortable tables/columns with units and descriptions, dataset provenance, and the canonical measurement-type registry, all reading the same `metadata.json` sidecar.

6.2 Versions

- **`latest` vs pinned.** `latest.txt` names the *promoted* version — a release is promoted only after its consumer-contract query suite passes, so `latest` is safe to follow interactively and is what `cc_get_db()` defaults to. Anything that must be reproducible — a paper, an app build, a download bundle — pins a `vYYYY.MM.DD`. Releases are immutable: the same version always returns the same rows.
- **Consolidated vs retired.** Every version keeps its sidecars and notes, but only some keep their bytes. The **consolidated** versions (`"consolidated": true` in `versions.json`) mark a schema or data milestone and keep their `parquet/` copy indefinitely — `v2026.04.08` (last of the per-dataset schema), `v2026.05.14` (first `ctd_thin`), `v2026.06.26` (zoodb and zooscan enter), `v2026.07.17` (unified `taxon`),

`v2026.08.14` (last before content-addressing) and `v2026.08.25` (`cruise_key` by date span, depth ceiling, quality flags mapped) — plus, always, the promoted version and its predecessor. Every other version is **retired** by archive thinning once it is neither: its `versions.json` entry gains `retired: {retired_utc, to, reason}`, where `to` is the consolidated version to use instead, and `cc_get_db("v2026.08.10")` errors naming it rather than failing table by table on 404s. When you pin, pin a consolidated version.

- **What changed.** RELEASES.md is the running changelog across versions — one section per release, newest first, saying what was wrong, what is true now, and what it costs a consumer (`**Consumers:**` lines). Each version's `RELEASE_NOTES.md` is its section plus a generated appendix (tables and rows, datasets, the consumer-contract result, package versions): `calcofi4r::cc_release_notes(version)` prints it, and `cc_list_versions()` / `calcofi4py.cc_list_versions()` list `versions.json`. Per table, `objects[].since` in the catalog says which version last changed its bytes.

6.3 Setup: the `httpfs` extension

DuckDB reads remote Parquet through its `httpfs` extension. Spatial queries (e.g. distances between casts and tows) also need `spatial`. Both are one-time installs, then loaded per session:

```
INSTALL httpfs; LOAD httpfs;
INSTALL spatial; LOAD spatial;   -- only if you use ST_* functions
```

6.4 Single-file tables

A single-file table is one HTTPS URL handed to `read_parquet()`. Take it from the catalog (the packages do this for you, below); typed by hand it must be a version whose `parquet/` copy is kept — `v2026.08.25` is consolidated, so this URL answers indefinitely:

```
SELECT taxon_key, scientific_name, common_name, worms_id, rank
FROM read_parquet('https://storage.googleapis.com/calcofi-db/ducklake/releases/v2026.08.25/pa
WHERE scientific_name = 'Sardinops sagax';
```

Joins work the same way — name each table's URL. The core model is small: `obs` holds one measured value per row (`realm` = `bio` | `env`) and points at the `sample` it came from (`sample_key`; `parent_sample_key` walks net → tow → site and bottle → cast) and, for biology, at `taxon` (`taxon_key`); event-level effort such as `std_haul_factor` is in `sample_measurement`:

```
SELECT s.sample_key, s.sample_type, s.datetime, s.latitude, s.longitude,
       sm.measurement_type, sm.measurement_value
FROM read_parquet('https://storage.googleapis.com/calcofi-db/ducklake/releases/v2026.08.25/pa
JOIN read_parquet('https://storage.googleapis.com/calcofi-db/ducklake/releases/v2026.08.25/pa
WHERE s.dataset_key = 'swfsc_ichthyo'
  AND sm.measurement_type = 'std_haul_factor'
LIMIT 10;
```

6.5 Hive-partitioned tables

`obs` (and the supplemental `obs_ctd_full` / `obs_mets_full`) is partitioned into one folder per `dataset_key`. From the v2026.09 releases the catalog lists each partition as its own object, so a partitioned table is read as an **explicit list of HTTPS URLs** — `objects[].path` prefixed with the bucket root — with `hive_partitioning = true` to recover `dataset_key` from the `dataset_key=…` path segment. That runs in every DuckDB, the browser included:

```
-- the list is catalog.json  tables[name = 'obs']  objects[].path, one per partition
SELECT dataset_key, count(*) AS n
FROM read_parquet([
    'https://storage.googleapis.com/calcofi-db/ducklake/tables/obs/dataset_key=calcofi_bottle
    'https://storage.googleapis.com/calcofi-db/ducklake/tables/obs/dataset_key=calcofi_ctd-ca
    -- … one URL per partition, in catalog order
    'https://storage.googleapis.com/calcofi-db/ducklake/tables/obs/dataset_key=swfsc_ichthyo/
  ], hive_partitioning = true)
GROUP BY dataset_key
ORDER BY dataset_key;
```

Because `dataset_key` is the partition column, filtering on it lets DuckDB **prune** whole partitions — a single-dataset query never opens the other files. Nobody types that list: `calcofi4r::cc_read_parquet_sql()` / `calcofi4py.read_parquet_sql()` emit it from the catalog (next sections).

A partitioned table may **also** publish one whole-table file — a single-file **twin** — and `obs` does. In the catalog it is the `objects[]` entry *without* `partition_by` (`ducklake/tables/obs/{content_hash}/obs.parquet`); the resolvers leave it out of `urls` and return it separately as `single_file` (`NA` / `None` for a table that has none). It exists for https-only readers that cannot take a list — browser DuckDB-WASM, a `read_parquet()` that wants one URL — which read `single_file` *instead of* the partition list. Read one or the other, **never both**: the twin holds the same rows as the partitions, so a query over both counts every row twice. The price of the twin is partition pruning — a `dataset_key` filter

still works, but scans the one file. On releases before v2026.09 it is the plain per-release copy, `…/releases/{version}/parquet/obs.parquet`:

```
-- the single-file twin of obs, for readers that cannot take a list or glob
SELECT dataset_key, count(*) AS n
FROM read_parquet('https://storage.googleapis.com/calcofi-db/ducklake/releases/v2026.08.25/pa
GROUP BY dataset_key
ORDER BY dataset_key;
```

Releases **before v2026.09** (`v2026.08.25` and the consolidated versions before it) have no `objects[]`: their partitioned table is a directory under `parquet/`, and plain HTTPS URLs cannot glob a directory. Reading one needs an **s3-style glob**, so configure DuckDB's anonymous s3 access — GCS is s3-compatible:

```
INSTALL httpfs; LOAD httpfs;
SET s3_region            = 'auto';
SET s3_endpoint          = 'storage.googleapis.com';
SET s3_url_style         = 'path';
SET s3_access_key_id     = '';
SET s3_secret_access_key = '';

SELECT dataset_key, count(*) AS n
FROM read_parquet(
  's3://calcofi-db/ducklake/releases/v2026.08.25/parquet/obs/**/*.parquet',
  hive_partitioning = true)
GROUP BY dataset_key
ORDER BY dataset_key;
```

The resolvers return this form for those versions automatically.

6.6 From R

Let the `calcofi4r` package register every release table as a view — it resolves each table through the catalog and handles `httpfs`, the partitioned tables, the supplemental opt-in and local caching (content-addressed, so a table unchanged between releases is cached once):

```
# remotes::install_github("calcofi/calcofi4r")
library(calcofi4r)

con <- cc_get_db()                      # promoted release, tables as views
```

```r
con <- cc_get_db("v2026.08.25")          # pinned; a retired version errors, naming the repla
DBI::dbListTables(con)

# lazy dbplyr against the remote parquet
library(dplyr)
tbl(con, "taxon") |> filter(scientific_name == "Sardinops sagax")

# or a one-off SQL query
cc_query("SELECT dataset_key, count(*) AS n FROM obs GROUP BY 1 ORDER BY 2 DESC")
```

To hand a table's URLs to DuckDB (or anything else) yourself, resolve them through the catalog rather than building a path — `cc_release_sources()` is the one place a catalog entry becomes URLs, for any version:

```r
library(DBI)
cat_ <- cc_catalog("latest")              # or a pinned "vYYYY.MM.DD"
src  <- cc_release_sources(cat_, "obs")   # list(urls, hive, canonical, hashes, local_paths,
src$urls                                  # one https URL per partition (the legacy path or s
src$single_file                           # obs's whole-table twin - read it OR src$urls, nev

con <- dbConnect(duckdb::duckdb())
dbExecute(con, "INSTALL httpfs; LOAD httpfs;")
d <- dbGetQuery(con, sprintf(
  "SELECT scientific_name, common_name, worms_id
   FROM %s
   WHERE common_name ILIKE '%%sardine%%'",
  cc_read_parquet_sql(cc_release_sources(cat_, "taxon"))))   # read_parquet('…') or read_parc
```

6.7 From Python

calcofi4py registers every release table as a DuckDB view for you (partitioned + supplemental handling included), mirroring `calcofi4r`:

```python
# pip install "calcofi4py @ git+https://github.com/CalCOFI/calcofi4py"
import calcofi4py as cc
con = cc.cc_get_db()                      # promoted release; cc_get_db("v2026.08.25") to pin
df  = con.sql("SELECT * FROM taxon WHERE common_name ILIKE '%sardine%'").df()
cc.cc_query("SELECT count(*) FROM obs").fetchone()
```

! Apply the quality flags

`obs.measurement_qual` carries each dataset's **own** flag vocabulary, uninterpreted — bottle (6 = OK but from CTD, 8 = suspect, 9 = missing), CTD cast files (1/2 = use the primary/secondary sensor, 8 = questionable, 9 = bad/missing), DIC WOCE (2 = good, 3 = questionable, 4 = bad, 9 = missing); see metadata/measurement_qual.csv. A flagged value is still a row. Filter it yourself, or use the one predicate the apps use:

```
calcofi4r::cc_qual_ok_sql("o")     # append to any WHERE over obs / obs_ctd_full / sample_m
```

```
calcofi4py.qual_ok_sql("o")
```

```
-- what both expand to (NULL-safe: an unflagged row is kept)
AND COALESCE(NOT ((o.dataset_key IN ('calcofi_bottle', 'calcofi_ctd-cast')
                       AND regexp_replace(o.measurement_qual, '\.0+$', '') IN ('8', '9'))
                  OR (o.dataset_key = 'calcofi_dic' AND o.measurement_qual IN ('3', '4', '9'))
```

Or plain `duckdb`, resolving the URLs through the catalog yourself (`release_sources()` mirrors `calcofi4r::cc_release_sources()`; a retired version raises `RetiredVersionError`, whose `.to` is the version to use instead):

```
import duckdb
import calcofi4py as cc

catalog = cc.cc_catalog("latest")                  # or "v2026.08.25"
src     = cc.release_sources(catalog, "taxon")     # {"urls", "hive", "canonical", "hashes",

con = duckdb.connect()
con.sql("INSTALL httpfs; LOAD httpfs;")
df = con.sql(f"""
  SELECT scientific_name, common_name, worms_id
  FROM {cc.read_parquet_sql(src)}
  WHERE common_name ILIKE '%sardine%'
""").df()
```

Without the package the same resolution is a few lines of `json`: fetch `releases/{version}/catalog.json` and prefix each table's `objects[].path` with `https://storage.googleapis.com/calcofi-db/`, as the browser example below does.

6.8 From your browser

DuckDB compiles to WebAssembly, so the *same engine* runs **client-side in your browser** — no server, no install, no R or Python. CalCOFI ships a ready-made form at → **Open CalCOFI Query** that wraps the three retired bio env Plumber endpoints (and a free-form SQL shell): pick a function, fill the form, click **Run**, and a Chromium / Firefox / Safari worker thread fetches Parquet range-by-range over HTTPS and joins them in-browser.

To embed DuckDB-WASM in your own page, the boilerplate is about 15 lines — load the bundle from a CDN, instantiate, `INSTALL httpfs` and you're ready:

```
<script type="module">
  import * as duckdb from "https://cdn.jsdelivr.net/npm/@duckdb/duckdb-wasm@1.29.0/+esm";

  const bundles = duckdb.getJsDelivrBundles();
  const bundle  = await duckdb.selectBundle(bundles);
  const worker  = await duckdb.createWorker(bundle.mainWorker);
  const db      = new duckdb.AsyncDuckDB(new duckdb.ConsoleLogger(), worker);
  await db.instantiate(bundle.mainModule, bundle.pthreadWorker);
  const conn = await db.connect();
  await conn.query("INSTALL httpfs; LOAD httpfs;");
  await conn.query("INSTALL spatial; LOAD spatial;");

  const root    = "https://storage.googleapis.com/calcofi-db";
  const version = (await (await fetch(`${root}/ducklake/releases/latest.txt`)).text()).trim()
  const catalog = await (await fetch(`${root}/ducklake/releases/${version}/catalog.json`)).js

  // resolve a table through the catalog: objects[].path → https URL. A partitioned
  // table lists one object per partition (needs hive_partitioning) and may also
  // publish a whole-table twin - the object WITHOUT partition_by (obs does); a
  // browser reads the twin instead of the list, never both (that doubles every row).
  // catalogs before v2026.09 have no objects[] - fall back to the per-release path
  const readParquet = (name) => {
    const t    = catalog.tables.find(t => t.name === name);
    const objs = t.objects ?? [{ path: `ducklake/releases/${version}/parquet/${name}.parquet
    const twin = t.partitioned ? objs.find(o => !o.partition_by) : null;
    if (twin) return `read_parquet('${root}/${twin.path}')`;
    const urls = objs.map(o => `'${root}/${o.path}'`).join(", ");
    return `read_parquet([${urls}]${t.partitioned ? ", hive_partitioning = true" : ""})`;
  };

  const result = await conn.query(`
    SELECT scientific_name, common_name, worms_id
```

```
    FROM ${readParquet("taxon")}
    WHERE common_name ILIKE '%sardine%'
  `);
  console.log(result.toArray());
</script>
```

For arbitrary one-off SQL with no setup at all, paste a query into **shell.duckdb.org** — DuckDB's official DuckDB-WASM shell. Same WebAssembly engine, same public Parquet, identical rows.

6.9 Run it from anywhere

The same portable SQL — what `calcofi4r::cc_match_*()` emits as `attr(d, "sql")`, what the Integrated App download bundle ships in its `query/` folder, what is shown in the worked example below — runs in any DuckDB client:

Where	How
R, on your laptop	`calcofi4r::cc_match_ichthyo_by_name(...)` — emits and runs the SQL; `attr(d, "sql")` hands it back
Python, on a notebook server	`duckdb.connect().sql(open("query.sql").read()).df()` — see From Python
shell, on the command line	`duckdb < query.sql`
your web browser, no install	**CalCOFI Query** — point-and-click form, runs DuckDB-WASM client-side

6.10 Reproducibility

Because every query is plain SQL against immutable, versioned, public Parquet, a CalCOFI result is **reproducible by anyone** — pin the `{version}` and re-run the SQL. Pin a consolidated version: its bytes are kept indefinitely, and a later release that shares an object with it shares the same bytes.

The Integrated App builds on this: its data **download bundle** ships a `query/` folder alongside the data:

```
data/original/{bio,env}.csv          ← query/{bio,env}.sql
data/integrated/integrated_*.csv     ← query/integrated_*.sql
```

`query/manifest.json`	`release version, filters, GCS source URLs, per-file row counts + md5 checksums`
`query/REPRODUCE.md`	`DuckDB-CLI / Python / R re-run snippets`

Each `*.sql` file is fully interpolated, GCS-URL-based, and copy-paste runnable (prefixed with the `INSTALL/LOAD` it needs). Re-run `query/integrated_*.sql` in DuckDB and you get back exactly the rows in the matching `.csv` — the `manifest.json` md5 checksums let you confirm it. The same SQL is what `calcofi4r::cc_match_bio_env()` executes and attaches as `attr(x, "sql")`, what the browser-based **CalCOFI Query** generates as its `SQL` panel, and what a hand-written query produces — **a single source of truth across R, Python, the CLI and JavaScript**.

6.11 Worked example: sardine larvae + temperature

The recurring example through these pages is *Pacific sardine (`Sardinops sagax`) larvae matched to CTD-bottle temperature, Q1 2018, with relaxed (5 km / 72 hr) matching.*

> **Note**
>
> Q1 **2018** — not a more recent year — because CTD-bottle environmental data in the current release ends 2021-05, while net-tow biological data runs later. Q1 2018 has ample overlap of both.

Done as **direct SQL**, the query is a temporal interval join plus a spatial `ST_Distance_Sphere` filter. After the `INSTALL/LOAD` setup above, run the query below. This block is character-for-character what `calcofi4r::cc_match_ichthyo_by_name()` returns as `attr(d, "sql")` with `version = "v2026.08.25"` — the two produce the **identical 13 rows**. Every table in it was resolved through that release's catalog, and because `v2026.08.25` is consolidated the URLs stay valid. On this pre-v2026.09 release the partitioned `obs` resolves to an `s3://` glob, so the query needs the anonymous-s3 settings; from v2026.09 the same call emits an explicit HTTPS list that runs anywhere, the browser included. The Integrated App download bundle builds `query/integrated_*.sql` with the same `cc_match_bio_env()` engine (its filter set is taxa + quarters + dates rather than `life_stage`, so a sardine / Q1 2018 bundle returns the egg + larva superset — same matching mechanics, same reproducibility):

```
WITH bio AS (
SELECT
  o.obs_id::VARCHAR AS bio_id,
  o.datetime AS bio_datetime,
  o.longitude AS bio_lon,
  o.latitude AS bio_lat,
```

```
  o.measurement_value * shf.measurement_value / nullif(ps.measurement_value, 0) AS bio_value
  o.measurement_value AS tally,
  o.taxon_key,
  t.scientific_name,
  t.worms_id,
  o.life_stage
FROM read_parquet('s3://calcofi-db/ducklake/releases/v2026.08.25/parquet/obs/**/*.parquet', h
JOIN read_parquet('https://storage.googleapis.com/calcofi-db/ducklake/releases/v2026.08.25/pa
LEFT JOIN read_parquet('https://storage.googleapis.com/calcofi-db/ducklake/releases/v2026.08
LEFT JOIN read_parquet('https://storage.googleapis.com/calcofi-db/ducklake/releases/v2026.08
WHERE o.realm = 'bio'
    AND o.dataset_key = 'swfsc_ichthyo'
    AND o.measurement_type = 'abundance'
    AND o.measurement_value IS NOT NULL
    AND COALESCE(NOT ((o.dataset_key = 'calcofi_bottle' AND regexp_replace(o.measurement_qual
    AND o.datetime IS NOT NULL
    AND o.longitude IS NOT NULL
    AND o.latitude IS NOT NULL
    AND t.scientific_name IN ('Sardinops sagax')
    AND o.life_stage IN ('larva')
    AND o.datetime >= TIMESTAMP '2018-01-01'
    AND o.datetime <= TIMESTAMP '2018-03-31'
),
env AS (
SELECT
  obs_id AS env_id,
  datetime AS env_datetime,
  longitude AS env_lon,
  latitude AS env_lat,
  measurement_value AS env_value,
  depth_min_m AS env_depth_m,
  measurement_type AS measurement_type
FROM read_parquet('s3://calcofi-db/ducklake/releases/v2026.08.25/parquet/obs/**/*.parquet', h
WHERE realm = 'env'
    AND measurement_type = 'temperature'
    AND measurement_value IS NOT NULL
    AND COALESCE(NOT ((dataset_key = 'calcofi_bottle' AND regexp_replace(measurement_qual, '
    AND datetime IS NOT NULL
    AND longitude IS NOT NULL
    AND latitude IS NOT NULL
    AND datetime >= TIMESTAMP '2018-01-01' - INTERVAL '72 hours'
    AND datetime <= TIMESTAMP '2018-03-31' + INTERVAL '72 hours'
```

```
),
matched AS (
  -- temporal interval join: every env observation within ± max_time_hr
  SELECT
    bio.*,
    env.* EXCLUDE (env_lon, env_lat),
    abs(epoch(bio.bio_datetime) - epoch(env.env_datetime)) / 3600.0 AS time_diff_hr,
    ST_Distance_Sphere(
      ST_Point(bio.bio_lon, bio.bio_lat),
      ST_Point(env.env_lon, env.env_lat)) / 1000.0                  AS dist_km
  FROM bio
  JOIN env
    ON env.env_datetime BETWEEN bio.bio_datetime - INTERVAL '72 hours'
                            AND bio.bio_datetime + INTERVAL '72 hours'
),
within AS (
  -- spatial filter: keep pairs within max_dist_km
  SELECT * FROM matched
  WHERE dist_km <= 5
),
ranked AS (
  SELECT
    *,
    min(time_diff_hr) OVER (PARTITION BY bio_id) AS mn_time_diff_hr,
    min(dist_km)      OVER (PARTITION BY bio_id) AS mn_dist_km
  FROM within
)
-- one row per bio observation (× measurement_type): env values aggregated
SELECT
  * EXCLUDE (
    env_id, env_value, env_datetime, env_depth_m,
    time_diff_hr, dist_km, mn_time_diff_hr, mn_dist_km),
  count(*)                                              AS n_env,
  avg(env_value)                                        AS env_value,
  CASE WHEN count(*) = 1 THEN 0
       ELSE coalesce(stddev_samp(env_value), 0) END     AS env_value_sd,
  avg(env_depth_m)                                      AS env_depth_m,
  min(env_datetime)                                     AS env_datetime_min,
  max(env_datetime)                                     AS env_datetime_max,
  avg(dist_km)                                          AS dist_km,
  avg(time_diff_hr)                                     AS time_diff_hr
FROM ranked
```

```
WHERE time_diff_hr = mn_time_diff_hr
GROUP BY ALL
ORDER BY bio_id
```

The next page, Matching Helpers, shows this same query as a one-liner.

7 Server Access

7.1 SSH, SFTP & PostgreSQL

The CalCOFI server (`ssh.calcofi.io`, a Google Cloud VM in Iowa) hosts the apps, RStudio Server, ERDDAP — and, since August 2026, a **multi-user PostgreSQL database** that the CTD team uses for QA/QC: everyone reads and writes the same tables, originals stay immutable, flags and proposed fixes live beside them. This page is the one place that explains how to get in and how to connect, for **Mac** and **Windows**.

> **Two databases, two access paths**
>
> - The **public releases** (`v2026.08.14` …) are Parquet on a public bucket. No account, no tunnel: Data Access, `calcofi4r::cc_get_db()`, DuckDB, the browser.
> - The **working PostgreSQL database** (`calcofi`) is private and reachable only over SSH. That is what the rest of this page is about.

7.2 The stack

Everything the server runs is one **Docker Compose stack**, configured in the CalCOFI/server repository — the config of record for every `*.calcofi.io` service (file an issue there for server problems):

service	container	serves
Caddy	`caddy`	reverse proxy + HTTPS for every subdomain below (`caddy/Caddyfile`)
Shiny Server	`rstudio`	the interactive apps at app.calcofi.io
RStudio Server	`rstudio`	rstudio.calcofi.io — R in the browser, on the server
PostgreSQL + PostGIS	`postgis`	the working `calcofi` database (below) and legacy `gis`

service	container	serves
pgAdmin	`pgadmin`	pgadmin.calcofi.io — web UI onto PostgreSQL
pg_tileserv	`pg_tileserv`	tile.calcofi.io — vector tiles from PostGIS
ERDDAP	`erddap`	erddap.calcofi.io — tabular data services
plumber	`plumber`	api.calcofi.io — the R API
Varnish	`varnish`	h3t.calcofi.io — cached hexagon-tile API (see Maps)

Caddy also fronts two static services with no container of their own: file.calcofi.io (public files on the server's disk) and storage.calcofi.io (a browsable front door onto the public Google Cloud Storage buckets).

7.3 Getting an account

Accounts are personal (no shared logins), key-only (no passwords over SSH), and named after your email local-part — `rswalethorp`, `bmgire`, `kdvogel`, `bthuang`, `esatterthwaite`, `bebest`. Each account gets:

what	where
SSH + SFTP login	`ssh.calcofi.io`, your home directory (small — keep big files under `~/ctd`)
shared CTD folder	`~/ctd` → `/share/data/ctd/{incoming,archive,exports}` (group-writable)
PostgreSQL role	same name as your username, in database `calcofi`; schemas `ctd` (curated), `work` (shared scratch), and your own schema (`rswalethorp.*`)
pgAdmin web login	pgadmin.calcofi.io with your email
RStudio Server login	rstudio.calcofi.io — R in the browser, already on the server (no tunnel needed there)

To request one: send Ben (bebest@ucsd.edu) your SSH *public* key. Generate it once per computer:

7.4 Mac (Terminal)

```
ssh-keygen -t ed25519 -C "you@ucsd.edu"      # accept the default file, set a passphrase
cat ~/.ssh/id_ed25519.pub                      # this ONE line is what you send
```

7.5 Windows (PowerShell)

Windows 10/11 ships OpenSSH (`ssh`, `ssh-keygen`, `sftp`, `scp`) — the commands are the same as on Mac. Open **PowerShell** (not cmd):

```
ssh-keygen -t ed25519 -C "you@ucsd.edu"      # accept the default file, set a passphrase
Get-Content $env:USERPROFILE\.ssh\id_ed25519.pub   # this ONE line is what you send
```

If `ssh-keygen` is not found: *Settings* *System* *Optional features* *Add* *OpenSSH Client*, then reopen PowerShell.

7.6 Windows (PuTTY)

If you already use PuTTY: **PuTTYgen** **Generate** (EdDSA/Ed25519), set a passphrase, **Save private key** (`calcofi.ppk`), and copy the text in *"Public key for pasting into OpenSSH authorized_keys file"* — that is what you send. (A `.ppk` can also be imported into the built-in OpenSSH with *Conversions* *Export OpenSSH key*.)

Never send the private key (`id_ed25519` without `.pub`). Lost laptop → tell Ben, the key is revoked and you send a new one.

7.7 Step 1 — SSH in

Put the server in your SSH config once, and every tool (ssh, sftp, R, VS Code) can use the short name `calcofi`. The `LocalForward` line is the database tunnel (next section).

7.8 Mac (Terminal)

Create or edit `~/.ssh/config` (e.g. `nano ~/.ssh/config`):

```
Host calcofi
  HostName ssh.calcofi.io
  User rswalethorp                  # <- your username
  IdentityFile ~/.ssh/id_ed25519
  LocalForward 5432 localhost:5432 # database tunnel (see below)
  ServerAliveInterval 60
```

Then:

```
ssh calcofi            # first time: answer "yes" to the host-key prompt
```

7.9 Windows (PowerShell)

Same file, at `C:\Users\<you>\.ssh\config` (create it with Notepad; no extension):

```
Host calcofi
  HostName ssh.calcofi.io
  User rswalethorp
  IdentityFile ~/.ssh/id_ed25519
  LocalForward 5432 localhost:5432
  ServerAliveInterval 60
```

```
ssh calcofi
```

7.10 Windows (PuTTY)

Session: Host Name `ssh.calcofi.io`, Port 22, Saved Sessions `calcofi`. **Connection Data**: Auto-login username = your username. **Connection SSH Auth Credentials**: Private key file = your `calcofi.ppk`. **Connection SSH Tunnels**: Source port `5432`, Destination `localhost:5432`, **Add**. Back on **Session**, **Save**, then **Open**. Keep the window open while you use the database. (Pageant can hold the key so you type the passphrase once per session.)

You land in your home directory. Useful first commands:

```
ls -la ~/ctd/          # the shared CTD folder (incoming/ archive/ exports/)
df -h /share           # how much disk is left - it is shared
cat ~/.pgpass          # your database password (Step 2)
```

VS Code

The **Remote – SSH** extension reuses the same `~/.ssh/config`: *Remote-SSH: Connect to Host… calcofi* gives you a file browser, terminal and editor on the server. Handy for looking at files under `/share/data/ctd` without SFTP.

7.11 Step 2 — your database password

Your PostgreSQL password was generated when the account was created and is stored **only** in a file in your home directory on the server — it was never emailed. Read it once:

```
cat ~/.pgpass
# host:port:database:user:password - the last field is your CalCOFI database password
# localhost:5432:*:rswalethorp:Xxxxxxxxxxxxxxxxxxxxxxx
# postgis:5432:*:rswalethorp:Xxxxxxxxxxxxxxxxxxxxxxx
```

`.pgpass` is the standard PostgreSQL password file: every PostgreSQL client (psql, R's `RPostgres`, Python's `psycopg`, pgAdmin desktop, DBeaver, DuckDB) reads it, so you never put the password in a script. Copy the `localhost` line into the same file **on your laptop**:

7.12 Mac

```
nano ~/.pgpass          # paste the localhost:5432:*:<you>:<password> line, save
chmod 600 ~/.pgpass     # required - libpq ignores the file if others can read it
```

7.13 Windows

The file is `%APPDATA%\postgresql\pgpass.conf` (usually `C:\Users\<you>\AppData\Roaming\postgresql\pgp`
Create the folder and the file with Notepad, one line: `localhost:5432:*:<you>:<password>`.
No `chmod` needed on Windows.

To change the password: `psql -h localhost calcofi -c '\password'` (through the tunnel), then update both `.pgpass` files. Until Google sign-in is enabled on pgAdmin, the **same password** is your pgadmin.calcofi.io password (login = your email).

7.14 Step 3 — the database over the tunnel

PostgreSQL is **not** exposed to the internet. Your SSH connection carries it: with the `LocalForward 5432 localhost:5432` line in place, *while* ***`ssh calcofi`*** *is open* (or `ssh -N calcofi`, which opens the tunnel without a shell), anything on your laptop that connects to `localhost:5432` is talking to the server's database.

```
ssh -N calcofi &                                  # tunnel only, in the background (Ctrl-C /
psql -h localhost -d calcofi                       # password from ~/.pgpass; user = your OS
psql -h localhost -d calcofi -U rswalethorp        # …or say it explicitly if they differ
```

Inside psql:

```
\dn                          -- schemas: ctd, work, public, yours
\dt ctd.*                    -- curated CTD tables
SHOW search_path;            -- "$user", work, ctd, public  (unqualified names land in YOUR sch
CREATE TABLE t AS SELECT 1 AS x;      -- -> rswalethorp.t, visible to colleagues
CREATE TABLE work.t2 AS SELECT 1;     -- -> work.t2, shared scratch, writable by all
```

> ⚠ "Address already in use" / port 5432 taken
>
> If `ssh` prints `bind [127.0.0.1]:5432: Address already in use`, something on your laptop (often a local Postgres or pgAdmin's bundled server) already owns 5432. Use **15432** instead in two places: `LocalForward 15432 localhost:5432` in `~/.ssh/config` (or the PuTTY Tunnels source port), and `port 15432` / `localhost:15432` wherever you connect (psql `-p 15432`, the `.pgpass` line, `PGPORT=15432` for R/Python).

7.14.1 Graphical clients

client	how	best for
pgAdmin 4 desktop (download)	*Register* → *Server*: General name `calcofi`; **Connection** host `localhost`, port `5432`, database `calcofi`, username *you*; **SSH Tunnel** tab: *Use SSH tunneling*, host `ssh.calcofi.io`, port 22, username *you*, authentication *Identity file* → your private key. It opens the tunnel itself — no terminal needed.	Windows users; browsing + query tool + ERD
pgadmin.calcofi.io (web)	log in with your email + database password (Google sign-in coming); servers *calcofi (CTD QA/QC)* and *gis (legacy 2022)* are already registered — on first connect replace the username with **yours** and enter your password (the master password it asks for is a local encryption key of your choosing)	zero install, any machine
DBeaver (free)	New connection → PostgreSQL → host `localhost`, db `calcofi`, user *you*; **SSH** tab → host `ssh.calcofi.io`, user *you*, *Public Key* auth	people who already use it
RStudio Server (rstudio.calcofi.io)	log in with your username + database password; in R use `host = "postgis"` (it is on the same network, no tunnel); `calcofi4r::cc_pg_connect()` works out of the box	R without installing anything

7.15 From R

`calcofi4r` 1.8.0 (`remotes::install_github("calcofi/calcofi4r")`) resolves the host, your role and the tunnel for you:

```
library(calcofi4r)
con <- cc_pg_connect()                        # tunnel already open; role + password from ~/.p
con <- cc_pg_connect(tunnel = TRUE)           # …or let it run `ssh -N calcofi` for you (needs
con <- cc_pg_connect(port = 15432, tunnel = TRUE)   # if you had to move the local port

DBI::dbListObjects(con, DBI::Id(schema = "ctd"))
library(dplyr)
tbl(con, I("ctd.cast")) |> filter(cruise_key == "2023-04-3322") |> collect()

# write to your own schema or to work - and read it back from any other client
DBI::dbWriteTable(con, DBI::Id(schema = "work", table = "my_check"), my_df, overwrite = TRUE)
DBI::dbDisconnect(con)
cc_pg_tunnel_close()                           # if you used tunnel = TRUE
```

Plain `RPostgres`, if you prefer:

```
con <- DBI::dbConnect(RPostgres::Postgres(),
  host = "localhost", port = 5432, dbname = "calcofi", user = "rswalethorp")  # password: ~/
```

PostGIS geometry comes through `sf`: `sf::st_read(con, query = "SELECT * FROM ctd.cast")`.

7.16 From Python

calcofi4py mirrors `calcofi4r` — same verbs, same defaults, password from the same `~/.pgpass`:

```
# pip install "calcofi4py @ git+https://github.com/CalCOFI/calcofi4py"
import calcofi4py as cc

con = cc.cc_pg_connect(tunnel=True)   # opens `ssh -N calcofi` for you; role + password from
con.execute("SELECT count(*) FROM ctd.cast WHERE is_best_stage").fetchone()

import pandas as pd
casts = pd.read_sql("SELECT * FROM ctd.v_scan_qc WHERE study = '2304SH' AND cast_id = '2304_
```

```
# propose a QC flag (curators accept/reject)
con.execute("""
  INSERT INTO ctd.flag (scan_id, variable, qual_code, reason)
  SELECT scan_id, 'temp1', 4, 'spike vs neighbours'
  FROM ctd.v_scan_best WHERE study=%s AND cast_id=%s AND depth=%s
""", ("2304SH", "2304_001d", 57))
con.commit()
cc.cc_pg_tunnel_close()
```

It also wraps the public releases (`cc.cc_get_db()` — DuckDB with every release table as a view, versions pinnable) and the bridge (`cc.cc_pg_attach()`, next section). Prefer plain libraries? psycopg / SQLAlchemy / GeoPandas read the same `.pgpass` — connection string `postgresql+psycopg://<you>@localhost:5432/calcofi`, no password in the URL.

7.17 DuckDB PostgreSQL

DuckDB's `postgres` extension lets **one query** join the public release Parquet with the team's PostgreSQL tables — and write back. From R:

```
con <- cc_get_db()                  # DuckDB with the release tables as views (public, no tunn
cc_pg_attach(con)                   # ATTACH the PostgreSQL db as `pg` (through your tunnel)
DBI::dbGetQuery(con, "
  SELECT c.cruise_key, count(*) AS n
  FROM pg.ctd.cast c
  JOIN sample s ON s.sample_key = c.sample_key      -- release table
  GROUP BY 1 ORDER BY 2 DESC LIMIT 5")
cc_pg_attach(con, alias = "pgw", read_only = FALSE) # writable: CREATE TABLE pgw.work.x AS SE
```

The same from Python (`calcofi4py`) or any DuckDB CLI:

```
import calcofi4py as cc
con = cc.cc_get_db(tables=["cruise", "sample"])   # release views
cc.cc_pg_attach(con)                               # + the PostgreSQL db as `pg`
con.sql("SELECT count(*) FROM pg.ctd.flag").fetchone()
```

Raw SQL, in any DuckDB:

```
INSTALL postgres; LOAD postgres;
ATTACH 'dbname=calcofi host=localhost port=5432 user=rswalethorp' AS pg (TYPE postgres, READ
SELECT * FROM pg.ctd.cast LIMIT 5;
-- v2026.08.14 is a consolidated release, so this per-release path is kept; for any
-- other version take the URL from the release catalog (see below)
SELECT * FROM read_parquet('https://storage.googleapis.com/calcofi-db/ducklake/releases/v202
```

And the other direction — DuckDB *inside* PostgreSQL. The server runs the pg_duckdb extension, so from psql / pgAdmin you can read the public release Parquet without leaving SQL:

```
SELECT * FROM release.cruise WHERE year = 2023;        -- views over the current release (cru
SELECT r['sample_key']::text, r['datetime_utc']::timestamp
FROM read_parquet('https://storage.googleapis.com/calcofi-db/ducklake/releases/v2026.08.14/pa
WHERE r['dataset_key']::text = 'calcofi_ctd-cast' LIMIT 10;
```

The `v2026.08.14` URL works because that version is **consolidated** — its `parquet/` copy is kept indefinitely — whereas a retired version's is removed by archive thinning (see Versions). For any other version, resolve the table through its `catalog.json` instead of typing a path; the packages hand back the `read_parquet(...)` fragment to paste in:

```
calcofi4r::cc_read_parquet_sql(cc_release_sources(cc_catalog("latest"), "sample"))
#> read_parquet('https://storage.googleapis.com/calcofi-db/ducklake/…/sample.parquet')
```

(`r['column']::type` is pg_duckdb's row syntax. Big tables — `obs`, `obs_ctd_full` — are better filtered in DuckDB on your laptop than pulled through the server.)

7.18 The CTD QA/QC database

Database `calcofi`, three kinds of schema:

schema	what	who writes
`ctd`	**curated**: the calcofi.org db-CSV cast files loaded verbatim and never edited (`ctd.file`, `ctd.scan` — all 82 source columns, ~10.8 M scans; `ctd.scan_issue` keeps the handful of cells that could not be typed, as text; `ctd.scan_column` is the data dictionary), the flag ledger (`ctd.flag`, IODE codes in `ctd.qual_code`), derived `ctd.cast`, and views: `ctd.v_scan_best` (one archive+stage per cruise × direction), `ctd.v_scan_qc` (originals + `<var>_qc` / `<var>_fix` from accepted flags), `ctd.v_scan_clean` (fixes applied, accepted-bad nulled)	originals: the loader only; flags: everyone proposes (`INSERT INTO ctd.flag (scan_id, variable, qual_code, reason, proposed_value)`), curators set `status` to accepted/rejected
`release`	read-only views over the **public DuckDB release** Parquet via pg_duckdb (`release.cruise`, `release.ship`, `release.dataset`)	nobody (regenerated per release)
`work`	shared scratch — anything goes, readable and writable by the whole team	everyone
`<you>`	your personal schema (first on your `search_path`); colleagues can read it	you

The rules of the road: **originals never change** — a problem is a row in `ctd.flag` (which scan, which variable, what code, optional proposed value, why), a fix is an *accepted* flag, and every derived product is computed from originals + accepted flags so it can be regenerated. A first flag, end to end:

```
-- propose: temperature spike on one scan (IODE 4 = bad); 5 = changed, with proposed_value
INSERT INTO ctd.flag (scan_id, variable, qual_code, reason)
SELECT scan_id, 'temp1', 4, 'spike vs neighbours' FROM ctd.v_scan_best
WHERE study = '2304SH' AND cast_id = '2304_001d' AND depth = 57;
-- review (curators): accept or reject; reviewed_by/at are filled in for you
UPDATE ctd.flag SET status = 'accepted', review_note = 'agree' WHERE flag_id = 123;
-- see it: the _qc column carries the code, v_scan_clean nulls the value
SELECT depth, temp1, temp1_qc FROM ctd.v_scan_qc WHERE study = '2304SH' AND cast_id = '2304_
```

Every change to `ctd.flag` is kept in `ctd.flag_audit`. The schema and vocabulary are new (August 2026) — propose changes in `work`, and they will be folded into `ctd`.

7.19 Files: SFTP and the shared folder

```
sftp calcofi                        # same key, same alias; cd ctd/incoming ; put file.zip ; g
scp big.zip calcofi:ctd/incoming/   # one-shot copy
```

GUI clients: **Cyberduck** (Mac/Windows), **FileZilla**, **WinSCP** (Windows; uses the `.ppk` or the OpenSSH key) — protocol SFTP, host `ssh.calcofi.io`, user *you*, key file.

`/share/data/ctd` is owned by group `calcofi` with the group-write bit inherited, so files you upload are editable by teammates. Put **uploads in `incoming/`**, leave `archive/` to the loader, and use `exports/` for things you want others (or yourself, later) to download. Your home directory is on the small system disk — keep it for dotfiles and scripts.

7.20 Etiquette & limits

- One 4-vCPU / 15 GB machine runs everything. `LIMIT` first, `\timing` on, and ask before a `CREATE INDEX` on the big `ctd.scan` table.
- Backups: nightly dump → Google Cloud Storage, weekly automated restore test. Still, a `DROP TABLE` is a `DROP TABLE` — `work` is scratch, your schema is yours, `ctd` is protected.
- Disk is shared and finite; delete what you no longer need under `~/ctd`.
- Problems, keys, password resets, new tables in `ctd`: Ben Best (bebest@ucsd.edu).

8 Matching Helpers

The `calcofi4r` package provides helper functions that relate **biological** observations (net-tow ichthyoplankton or zooplankton biomass) to **environmental** observations (CTD-bottle measurements) by matching them in time and space — on the fly, against the public release Parquet on GCS.

These supersede the retired Postgres Plumber API endpoints `zooplankton_biomass`, `itis_ichthyodata` and `ichthyodata`, which depended on pre-built `uunet2ctd*` match tables that are no longer part of releases.

8.1 Install

```
# remotes::install_github("calcofi/calcofi4r")
library(calcofi4r)
```

The matching helpers need `calcofi4r` 1.2.0 and DuckDB with the `httpfs` and `spatial` extensions (the functions install/load these for you).

8.2 The wrappers

Three wrappers cover the common cases. Each builds the matching SQL, runs it, and returns a tibble of **one row per biological observation** with the matched environmental value:

Function	Biological side	Replaces
`cc_match_ichthyo_by_name()`	ichthyoplankton, filtered by scientific name	`/ichthyodata`
`cc_match_ichthyo_by_taxon()`	ichthyoplankton, filtered by a WoRMS taxon **and all its descendants**	`/itis_ichthyodata`
`cc_match_zooplankton_biomass()`	net-tow displacement-volume biomass (`totalplankton` / `smallplankton`)	`/zooplankton_biomass`

```r
# ichthyoplankton by scientific name
d <- cc_match_ichthyo_by_name(
  "Sardinops sagax", env_var = "temperature")

# every descendant of a WoRMS taxon (recursive parentNameUsageID walk)
d <- cc_match_ichthyo_by_taxon(
  worms_id = 125724, env_var = "salinity")   # 125724 = genus Engraulis

# zooplankton biomass
d <- cc_match_zooplankton_biomass(
  env_var = "temperature", biomass_type = "totalplankton")
```

8.3 Worked example: sardine larvae + temperature

The recurring example through these pages — *Pacific sardine (**Sardinops sagax**) larvae matched to CTD-bottle temperature, Q1 2018, with relaxed (5 km / 72 hr) matching* — is a single call:

```r
d <- cc_match_ichthyo_by_name(
  "Sardinops sagax",
  env_var        = "temperature",
  life_stage     = "larva",
  date_min       = "2018-01-01",
  date_max       = "2018-03-31",
  relax_matching = TRUE)
```

```r
d[, c("scientific_name", "life_stage", "bio_datetime",
      "bio_value", "n_env", "env_value", "dist_km", "time_diff_hr")]
#> # A tibble: 13 × 8
#>   scientific_name life_stage bio_datetime        bio_value n_env env_value dist_km time_di
#>   <chr>           <chr>      <dttm>                  <dbl> <dbl>     <dbl>   <dbl>
#> 1 Sardinops sagax larva      2018-02-05 00:44:00     116.     34      11.4   0.167
#> 2 Sardinops sagax larva      2018-02-05 06:03:00      17.8    30      10.5   0.225
#> 3 Sardinops sagax larva      2018-02-07 09:56:00     120.     33      10.4   …
#> # … 10 more rows
```

`bio_value` is the standardized larval tally; `env_value` is the matched mean temperature (°C); `n_env` is how many bottle measurements were averaged; `dist_km` / `time_diff_hr` are the realized match offsets (within the 5 km / 72 hr window). This returns the **identical 13 rows**

as the direct SQL — `attr(d, "sql")` is that exact query. The Integrated App download bundle uses the same `cc_match_bio_env()` engine for its `query/` folder.

> **Note**
>
> Q1 **2018**, not a more recent year: CTD-bottle environmental data in the current release ends 2021-05, while net-tow biological data runs later. Q1 2018 has ample overlap.

8.4 Reproducible SQL

Every helper attaches the **exact, portable SQL** that produced the result, plus query metadata, as attributes:

```
cat(attr(d, "sql"))           # the fully-interpolated, GCS-URL-based query
str(attr(d, "query_meta"))    # package + release version, params, source URLs
```

`attr(d, "sql")` is character-for-character the query shown on the Data Access page — copy-paste it into the DuckDB CLI, Python, a colleague's R session, or the browser-based **CalCOFI Query** and it returns identical rows. That is the contract behind the Integrated App download bundle's `query/` folder.

To get the SQL **without running it** — e.g. to serialize or inspect it — pass `return_sql = TRUE`:

```
sql <- cc_match_ichthyo_by_name("Sardinops sagax", return_sql = TRUE)
cat(sql)
```

8.5 Run it from anywhere

The portable SQL these helpers emit runs in any DuckDB client:

Where	How
R, on your laptop	`cc_match_ichthyo_by_name(...)` — this page
Python, on a notebook server	`duckdb.connect().sql(open("query.sql").read()).df()` — see Data Access → From Python
shell, on the command line	`duckdb < query.sql`

Where	How
your web browser, no install	**CalCOFI Query** — point-and-click form for the three wrappers + custom SQL + a free-form SQL shell, all running DuckDB-WASM client-side

In each case the *exact same* generated SQL hits the *exact same* public GCS Parquet, so a colleague who only writes Python (or only opens browser tabs) gets the same rows you do — same `bio_ids`, same `env_values`, same `time_diff_hrs`.

8.6 The core engine: `cc_match_bio_env()`

The wrappers are thin shells over `cc_match_bio_env()`, which takes a `bio` and an `env` SQL `SELECT` string and performs the match. Call it directly when you need a biological or environmental side the wrappers don't cover (custom filters, a different grouping, joining `bottle` to `cast_condition`, …):

```
cc_match_bio_env(
  bio, env,                                  # SELECT strings (see contract below)
  max_dist_km = 2, max_time_hr = 6,          # match tolerances
  join_method = "nearest_time",              # or "nearest_dist", "average"
  version     = "latest",
  collect     = TRUE,                        # FALSE → lazy dplyr::tbl
  return_sql  = FALSE)                       # TRUE  → just the SQL string
```

Contract. `bio` must yield `bio_id`, `bio_datetime`, `bio_lon`, `bio_lat`, `bio_value` (plus any descriptive columns, carried through as grouping keys). `env` must yield exactly `env_id`, `env_datetime`, `env_lon`, `env_lat`, `env_value`, `env_depth_m`, `measurement_type`. Both typically `FROM read_parquet('https://…/{table}.parquet')` so the emitted SQL stays portable.

8.7 Parameters

- **`max_dist_km`, `max_time_hr`** — the spatial and temporal match windows. Defaults are 2 km / 6 hr; `relax_matching = TRUE` widens them to 5 km / 72 hr (an explicit value always wins). This replaces the old API's `relax_matching` boolean.
- **`join_method`** — how the environmental observations inside the window are reduced to one value per biological observation:

  - `"nearest_time"` (default) — keep the closest in time, average ties;
  - `"nearest_dist"` — keep the closest in space, average ties;
  - `"average"` — average every observation in the window.

  CalCOFI co-locates net tows and CTD casts, so in practice `nearest_time` and `nearest_dist` usually agree and `average` differs only slightly.
- **`life_stage`** (ichthyo wrappers) — restrict to e.g. `"larva"` or `"egg"`.
- **`date_min`, `date_max`** — bound the biological side by tow start date.
- **`depth_m_min`, `depth_m_max`** — bound the environmental bottle depths.
- **`version`** — a release string like `"v2026.05.14"`, or `"latest"`.

8.8 Migrating from the API

Old API	New helper
`/ichthyodata` (`species`, `exact_match`)	`cc_match_ichthyo_by_name(scientific_name, exact_match=)`
`/itis_ichthyodata` (`ITISid`)	`cc_match_ichthyo_by_taxon(worms_id)` — WoRMS, recursive subtree
`/zooplankton_biomass`	`cc_match_zooplankton_biomass()`
`relax_matching = TRUE`	`relax_matching = TRUE` (5 km / 72 hr) or explicit `max_dist_km` / `max_time_hr`
`cruiseymd_min` / `cruiseymd_max`	`date_min` / `date_max`
`stage`	`life_stage`

The taxon wrapper is the one real change: ITIS `path`-regex matching is gone; descendants are now resolved by a recursive walk of WoRMS `taxon.parentNameUsageID` against the release `taxon` table.

9 API

! Superseded by direct querying + `calcofi4r` helpers

The Postgres-backed Plumber API at api.calcofi.io is **being phased out**. CalCOFI data is now published as versioned, public **Parquet** files that you query directly with DuckDB — no API server, no credentials, no rate limits.

- To query the data directly (SQL, R, Python), see **Data Access**.
- For biological environmental matching (the old `zooplankton_biomass`, `itis_ichthyodata` and `ichthyodata` endpoints), see **Matching Helpers** — the `calcofi4r` functions `cc_match_zooplankton_biomass()`, `cc_match_ichthyo_by_taxon()` and `cc_match_ichthyo_by_name()`.

The endpoint notes below are retained as a historical reference and a map from each API endpoint to its replacement.

Try the new client-side query playground

The three retired bio env endpoints — and many more queries besides — are now an interactive form at **calcofi.io/db-query/** that runs entirely in your browser. Pick a query from the left-side accordion, fill the form, click **Run**, see the rows — no server, no credentials, no install. DuckDB-WASM in a worker thread, public Parquet on Google Cloud Storage, the same portable SQL the `calcofi4r::cc_match_*()` helpers emit.

The raw interface to the legacy Application Programming Interface (API) is available at:

- api.calcofi.io

9.1 Endpoint → replacement

Legacy endpoint	R helper (`calcofi4r`)	Browser
`/variables`	`cc_list_measurement_types()`	Query → measurement types
`/timeseries`	direct SQL aggregation — see Data Access	Query → SQL shell

Legacy endpoint	R helper (`calcofi4r`)	Browser
`/cruises`	`cc_read_cruise()`	Query → cruises
`/raster`	direct SQL + `pts_to_rast_idw()`	—
`/cruise_lines`, `/cruise_line_profile`	direct SQL against `ctd_cast` / `ctd_thin`	Query → SQL shell
`zooplankton_biomass`	`cc_match_zooplankton_biomass()`	**Query → zoo biomass**
`itis_ichthyodata`	`cc_match_ichthyo_by_taxon()`	**Query → by taxon**
`ichthyodata`	`cc_match_ichthyo_by_name()`	**Query → by name**

9.2 Legacy endpoint reference

9.2.1 `/variables`: get list of variables for timeseries

Get list of variables for use in `/timeseries`.

9.2.2 `/species_groups`: get species groups for larvae

Not yet working. Get list of species groups for use with variables `larvae_counts.count` in `/timeseries`.

9.2.3 `/timeseries`: get time series data

9.2.4 `/cruises`: get list of cruises

Get list of cruises with summary stats as CSV table for time (`date_beg`).

9.2.5 `/raster`: get raster map of variable

Get raster of variable.

9.2.6 `/cruise_lines`: get station lines from cruises

Get station lines from cruises (with more than one cast).

9.2.7 `/cruise_line_profile`

Get profile at depths for given variable of casts along line of stations.

10 Portals

10.1 Overview

CalCOFI data is available through various portals, each serving different purposes and user needs. This document outlines the main access points and their characteristics.

10.2 Data Flow

While it would be ideal for CalCOFI data to be available through a single portal, each portal has its strengths and limitations. The following diagram illustrates one possible realization of data flow between CalCOFI data and the portals: from raw data to the integrated database to portals and meta-portals.

In practice, CalCOFI is a partnership with various contributing members, so the authoritative dataset might flow differently, such as from EDI to the database to the other portals. The other portals, such as OBIS or ERDDAP, serve different audiences or purposes. The meta-portals like ODIS and Data.gov then index these portals to provide broader discovery of CalCOFI datasets.

10.3 Portals

While some portals serve as data repositories, others provide advanced data access and visualization tools. The following sections describe the main portals where CalCOFI data is available and their key features.

10.3.1 EDI

Environmental Data Initiative

- Complete dataset archives using DataOne software and EML metadata
- DOIs issued for all datasets ensuring citability
- Full archive allowing for any data file types
- Basic spatial and temporal filtering through web interface

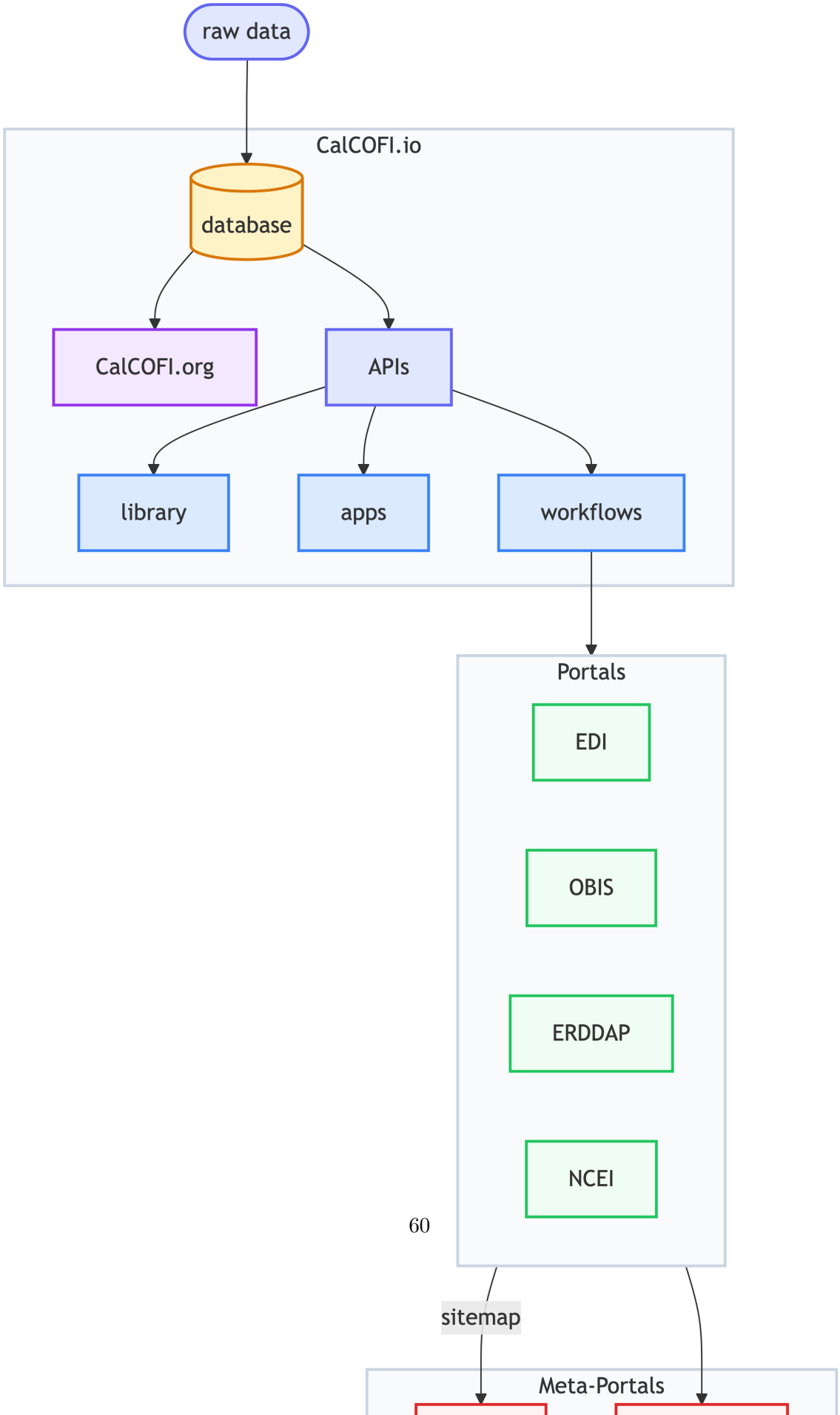

raw data
CalCOFI.io
database
CalCOFI.org
APIs
library
apps
workflows
Portals
EDI
OBIS
ERDDAP
NCEI
sitemap
Meta-Portals

Table 10.1: Portal Capabilities.

	Full Archive	Versioning	DOI Issued	Query by xyt	Query by taxa	Multiple formats
EDI						
NCEI						
OBIS						
ERDDAP						

Capability Legend: = full, = partial, = none

- Download in original formats with metadata
- Access through DataOne API
- Links:
  - EDIrepository.org
  - CalCOFI datasets: EDI query "CalCOFI"

10.3.2 NCEI

National Centers for Environmental Information

- Long-term archival of oceanographic data
- DOIs issued for dataset submissions
- Standardized metadata using ISO 19115-2
- Basic search interface with geographic and temporal filtering
- Data preserved in original submission formats
- Access through NCEI API services
- Links:
  - NCEI Ocean Archive
  - CalCOFI datasets: NCEI search "CalCOFI"

10.3.3 OBIS

Ocean Biodiversity Information System

- Specialized in marine biodiversity data
- Standardized using DarwinCore fields
- Extended measurements supported via extendedMeasurementOrFact
- Powerful filtering by space, time, and taxonomic parameters
- Multiple download formats (CSV, JSON, Darwin Core Archive)
- Full REST API access

- Links:
  - OBIS.org
  - CalCOFI datasets: obis.org/dataset + "calcofi" Keyword

10.3.4 ERDDAP

Environmental Research Division Data Access Program

- Tabular and gridded data server
- Advanced subsetting by space, time, and parameters
- Multiple output formats (CSV, JSON, NetCDF, etc.)
- RESTful API with direct data access
- Built-in data visualization tools
- No persistent identifiers but stable URLs
- Links:
  - ERDDAP
  - CalCOFI datasets:
    - ERDDAP, OceanView - CalCOFI seabirds
    - ERDDAP, CoastWatch - CalCOFI oceanographic

10.4 Metadata

The Ecological Metadata Language (EML) (and using R package EML in workflows) serves as a key standard for describing ecological and environmental data. For CalCOFI, EML metadata files are generated alongside data files, providing structured documentation that enables interoperability across different data portals. This metadata-driven approach allows automated ingestion into various data systems while maintaining data integrity and provenance.

The EML specification provides detailed structure for describing datasets, including:

- Dataset identification and citation
- Geographic and temporal coverage
- Variable definitions and units
- Methods and protocols
- Quality control procedures
- Access and usage rights

This standardized metadata enables automated data transformation and ingestion into various portal systems while preserving the original data context and quality information.

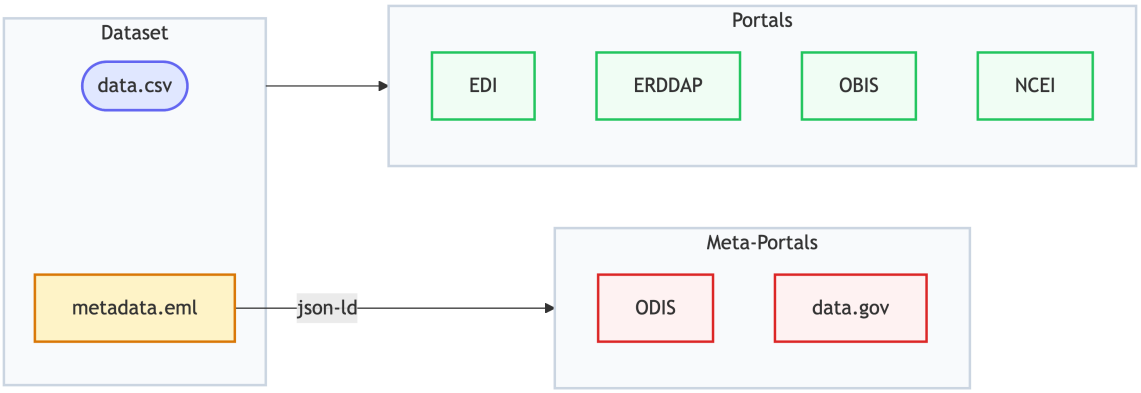


Figure 10.2: Metadata in the form of ecological metadata language (EML) is used to describe the dataset in a consistent manner that can be ingested by the portals.

10.5 Meta-Portals

10.5.1 Google Dataset Search

The JSON-LD metadata in the Portal dataset web pages get indexed by Google Dataset Search through schema.org metadata. This ensures that CalCOFI data is discoverable through Google search and other search engines.

10.5.2 ODIS

Ocean Data Information System

ODIS uses the same technology as Google Dataset Search (schema.org, JSON-LD), but focuses on ocean data. CalCOFI curates a sitemap of authoritative datasets to server to ODIS.org

This federated approach ensures that CalCOFI data remains:

- Discoverable through multiple channels
- Properly cited and attributed
- Integrated with global ocean data systems

10.6 CalCOFI.io Tools

CalCOFI is also developing an integrated database and tools that enable efficient data access and analysis:

10.6.1 APIs

- RESTful endpoints for programmatic access
- Filtering by space, time, and taxonomic parameters
- Relationship queries across tables
- Links:
  - api.calcofi.io
  - tile.calcofi.io

10.6.2 Library

- Direct data access from R
- Built-in analysis functions
- Integration with tidyverse ecosystem
- Link:
  - calcofi.io/calcofi4r

10.6.3 Apps

- Interactive data exploration with Shiny applications
- User-friendly interfaces
- Subset and download data
- Link:
  - calcofi.io, App button

11 Status

11.1 2026-07-01

This report covers the second half of the **2025-10-01 to 2026-06-30** contract period (work since the 2025-12-01 status) and serves as the end-of-contract wrap-up. It is organized around the three contract components — **Ingest**, **Visualize**, and **Share** — and closes with a deliverables crosswalk against the Statement of Work. Headline outcomes: all eight must-complete (**) datasets are ingested into a versioned, reproducible database; nine releases were published (v2026.03.25 → v2026.06.08, now 44 tables); a new on-the-fly hexagon tile service powers the integrated app; and CalCOFI data are now flowing to ERDDAP and OBIS, browsable through new schema and query explorers.

11.1.1 1. Ingest — A Versioned, Reproducible Integrated Database

The bulk of the period's effort went into the ingestion pipeline (`workflows`) and its engine package (`calcofi4db`), which together turn heterogeneous source files into a single, versioned database.

- **All must-complete datasets ingested.** Building on bottle, fish eggs & larvae (ichthyo), and CTD casts, the team added **DIC** (dissolved inorganic carbon), **phytoplankton** (Venrick, EDI #28, region-pooled grain with WoRMS taxonomy), **lobster phyllosoma** (EDI #33), **CUFES** (#35), **euphausiids/krill**, **PIC zooplankton**, and **bird & mammal census** data. Cephalopod/invertebrate counts were folded into the ichthyo dataset rather than maintained separately. Release **v2026.06.08** now spans **44 tables**.
- **Frozen, reproducible releases (DuckLake + Parquet on GCS).** `release_database.qmd` acts as the pipeline "caboose": it validates, freezes, and uploads each release to a Google Cloud Storage bucket as Parquet, with an auto-generated entity-relationship diagram and a `metadata.json` sidecar carrying full table/column descriptions, units, and data types. Nine releases were cut between March and June 2026.

- **A hardened, well-keyed data model.** Identifiers were standardized on stable keys — UUID-first source keys, `cruise_key` as `YYYY-MM-NODC`, and `sta_key → site_key`. The CTD data — by far the largest input — was tamed with a new adaptively-thinned `ctd_thin` table (5.5M rows / ~258 MB, preserving inflections via Douglas-Peucker) replacing the 15+ GB raw measurement table as the headline product, and several duplicate-key and phantom-cruise bugs were fixed.
- **`calcofi4db` formalized (v2.4 → v2.8).** New capabilities include content-hash deduplication of Parquet uploads (ignoring provenance columns), YAML-authoritative dataset metadata, native `GEOMETRY` storage with `ST_Hilbert` spatial ordering, dependency VIEWs, and server-side GCS copy — which together cut a full pipeline run from 60+ minutes to ~4 minutes. A reusable set of **Claude Code ingest skills** (explore → generate-metadata → ingest → validate) and **cross-dataset match helpers** now make adding a new dataset largely turnkey.
- **Data-lake migration.** Source contributions were migrated from a personal Google Drive to the organizational **CalCOFI Data Folder** shared drive, with `rclone`-based GDrive→GCS sync via a dedicated `calcofi-admin` service account (provisioned with Erin's help), establishing the long-term archive.

11.1.2 2. Visualize — On-the-Fly Hexagons, Management Areas, and Partner Dashboards

- **On-the-fly hexagon tile service (the cached-endpoint deliverable).** The integrated app previously pre-computed hexagon summaries for all zoom levels across the entire study area. A new **H3T tile API** now summarizes only the current zoom level and map extent on demand. It was prototyped as an R/Plumber service (`api-h3t`), then rewritten as a FastAPI service (`api-h3t-py`) with multi-database support, fronted by Varnish caching and edge gzip/zstd compression at `h3t.calcofi.io`. The integrated app reads hex data from it behind a `USE_H3T` flag.
- **Management areas of interest.** The integrated app (`db-viz-hex`) gained spatial layers for maritime zones, marine protected areas, and wind energy areas, plus a dark/light theme, reproducible query/data download bundles, and a direct connection to the `calcofi4r` Parquet database (replacing the old API dependency).
- **Standalone and partner apps.** New/redesigned Shiny apps include `ctd-viz` (linked map/table, plotly transects, GEBCO bathymetry), a `datacheck` cross-dataset observation explorer with deep-linkable URLs, and a **cruises** explorer. Three 2026 student-capstone visualization products were also supported and now ship from GitHub Pages: the **UCSB Station Data Portal**, the **UCSB Larvae Dashboard**, and the **UCLA**

California Ocean & Coastal Monitoring Inventory map — extending the partner coalition envisioned in the SOW.

11.1.3 3. Share — Publishing to ERDDAP & OBIS, plus Discovery Tools

- **ERDDAP.** A CalCOFI-branded ERDDAP server was configured and deployed, serving multiple `EDDTableFromParquetFiles` datasets (bottle, phytoplankton, and others). A dedicated benchmark study compared Parquet, NetCDF (`EDDTableFromNcCFFiles`), and DuckDB serving backends to choose the best layout — prompted by the wide CTD table OOM-ing ERDDAP's heap, which remains the one open serving issue (see crosswalk).
- **OBIS.** The fish eggs & larvae → OBIS workflow (Darwin Core + EML, joining biological observations to CTD environment by cruise) was refined to use the stable UUID keys and ichthyo-only scope.
- **Discovery & documentation.** Two new browser-based, serverless tools were built and linked throughout the docs: a **Schema explorer** (`schema/` — diagram, tables, columns, datasets, measurements, with a cross-tab dataset filter and clickable ERD) and a **Query playground** (`query/` — a Jekyll + DuckDB-WASM SQL sandbox). `calcofi4r` was rewired to read release `metadata.json`, emits `llms.txt`, and documents bio environment matching helpers; the `docs` site added Data Access and Matching pages.
- **Webinar series — pending.** The final Share deliverable (a recorded webinar series) has not yet been produced; a progress deck (Dec 2025 – Jun 2026) was assembled in its place.

11.1.4 Deliverables Crosswalk

Against the SOW (** = must-complete by end of contract):

Ingest — all must-complete datasets done:

Dataset	Status
Bottle Database **	ingested
Fish Eggs & Larvae **	ingested (ichthyo)
CTD Cast Files **	ingested (`ctd_thin` + `ctd_cast`)
Krill / euphausiids **	ingested

Dataset	Status
Zooplankton biovolume **	ingested (PIC)
Cephalopod / invertebrate **	folded into ichthyo
Phytoplankton (Venrick) **	ingested (#28)
DIC **	ingested
CUFES, lobster phyllosoma, bird/mammal census	ingested (beyond must-complete)

Visualize:

Deliverable	Status
Add management areas (sanctuaries, MPAs, wind areas)	added to db-viz-hex
On-the-fly hexagon endpoint by extent/zoom	H3T tile API live
Expand hex summaries across new datasets	in progress as datasets land

Share:

Deliverable	Status
CTD Cast Files → ERDDAP **	wide-table OOM under investigation (backend benchmark)
Bottle Database → ERDDAP **	published
Fish Eggs & Larvae → OBIS **	workflow complete
Other portals (EDI, NCEI, additional OBIS)	/ as prioritized
Recorded webinar series	pending (progress deck delivered)

11.1.5 Program Coordination

Beyond code, the period included sustained coordination with the CalCOFI program (Erin Satterthwaite and team): contributing to the **Data Management Plan**, gathering input on a **dataset search tool** (categorizing variables by EOVs), scoping **CalOOS/SCCOOS portal** integration, mentoring the UCSB/UCLA **capstone teams**, and contributing to an MBON ocean-indicators manuscript and several joint proposals (GOMO AI/Data pilots, IOOS OTT). EcoQuants LLC's CalCOFI work transitioned to **Ocean Metrics LLC** during this period.

11.2 2025-12-01

Over the past several months, CalCOFI's software and data systems have advanced significantly toward a unified, reliable, and user-friendly platform for exploring and publishing CalCOFI data. Work has focused on four main areas: the integrated data app, the underlying database and workflows, pushing to OBIS, and organizing of CTD data.

11.2.1 A More Capable, User-Friendly Integrated App

The **CalCOFI integrated application (`db-viz-hex`)** has evolved into a much richer and more intuitive tool for scientists and partners:

- **Taxonomy-aware exploration**
  The app now understands species and their taxonomic relationships. Users can browse taxa hierarchies, see taxonomic ranks, and work with improved species metadata tied to authoritative sources (e.g., WoRMS, ITIS). This makes it easier to find and compare species and groups of species consistently.
- **Better visual experience and theming**
  A new **dark/light theme toggle** has been implemented and refined so that maps, time series, and other plots remain readable and visually consistent. Navigation has been reorganized, with a clearer About page, a guided "tour" of the app, and more intuitive icons and labels, making the app easier to learn and use.
- **Stronger spatial and temporal tools**
  Spatial maps now rely on efficient hexagon grids calculated in the database, improving performance and scalability. Default settings for time and depth matching have been tuned to yield better joins between environmental and biological data out of the box.

Overall, the app is moving from a prototype to a **polished, guided interface** that better supports exploratory analysis and communication.

11.2.2 A Stable, Well-Documented Database Foundation

The **CalCOFI database package (`calcofi4db`)** has been formalized and versioned, providing a solid foundation for all downstream tools:

- **Two stable releases** (versions 1.0 and 1.1) have established a reliable baseline for the database, including bottle-level data.
- The project now follows a **clear strategy for separate development and production databases**, reducing risk when making changes and improving reproducibility.

- Data ingestion from NOAA and other sources has been **hardened**, with several rounds of fixes to handle edge cases and ensure that raw files are consistently and correctly translated into the database.

In addition, the package's **online documentation site** has been refreshed so that developers and analysts have up-to-date guidance on how data flow into and through the database.

11.2.3 Unified R Tools and Documentation Around DuckDB

Across the toolchain, CalCOFI has standardized on **DuckDB** as the core data engine:

- The **R package (`calcofi4r`)** now encapsulates key logic originally developed inside the integrated app, so the same high-quality data access and processing is available in scripts, reports, and analyses—not just in the web interface.
- Both the app and R package can connect to **local or remote DuckDB databases**, improving performance and enabling offline or near-offline workflows.
- Documentation in the **`docs` repository** has been updated to describe the full data creation process (from raw data to ready-to-use databases) and to explain the new development/production database strategy. Status documents and helper scripts provide clearer visibility into project progress.

This brings CalCOFI closer to a **coherent, documented platform** where analysts can move seamlessly between app-based exploration and scripted analysis.

11.2.4 Standards-Compliant Publication of Biological Data

The **`workflows`** repository has seen major progress in turning CalCOFI's biological datasets into **publication-ready products**:

- New workflows now **publish larval data to OBIS**, the global biodiversity information system, with repeatable recipes that combine biological observations with CTD (oceanographic) data.
- The underlying data model for **events and occurrences** has been strengthened to align with international standards (e.g., Darwin Core), including:
  - Clear hierarchies of sampling events,
  - Better handling of life-stage and size information (e.g., egg and larval stages),
  - Automated generation of metadata files required for data archives.

- Additional integrity checks and foreign key relationships help ensure that data are correct and consistent before publication.

These advances substantially improve CalCOFI's ability to **share high-quality, well-structured biodiversity data** with the broader scientific community.

11.2.5 Improved Public Access and Infrastructure

Finally, several changes improve how external users find and access CalCOFI tools:

- The **public website (`CalCOFI.github.io`)** now highlights key applications, including the integrated app and a pollutants-focused app, making them easier to discover.
- Server configuration has been updated so that **Shiny apps are served from a new dedicated domain `app.calcofi.io`** (and still `shiny.calcofi.io`), clarifying the entry point for interactive tools and simplifying operations.

11.2.6 Overall Impact

Together, these developments move CalCOFI toward a **modern, integrated data platform**:

- Scientists and partners gain a more powerful, user-friendly app and R toolkit for exploring CalCOFI data.
- The underlying database and workflows are more robust, testable, and clearly documented.
- CalCOFI's biological data are better positioned for **global visibility and reuse** through standards-compliant publication channels like OBIS.
- Public-facing web presence and infrastructure are cleaner and more aligned, making it easier for stakeholders to find and use CalCOFI resources.

11.3 2025-07-01

This report summarizes the key development activities, major accomplishments, and ongoing work for the first 6 monhts of 2025 across the CalCOFI GitHub repositories: **api, apps, calcofi4db, calcofi4r, docs, server, workflows**. The findings are based on issues and commits from January–July 2025.

11.3.1 API Enhancements

11.3.1.1 New Features & Data Integration

- **Expanded API Options**
  - Added ability to include bottle data and use relaxed criteria for net-to-cast matching (commit).
  - Supported upcast/downcast data downloads (commit).
  - Added Zooplankton biomass and improved ichthyodata output (commit).
- **Performance & Maintenance**
  - Implemented docker compose restart for Plumber API service (commit).
- **Ongoing Work**
  - Migration of database contouring functions to API/app level for improved caching and rendering efficiency.
  - Development of a robust, user-friendly API for seamless DB integration (issue).

11.3.2 Apps Development

11.3.2.1 Visualization & User Interface

- **Continuous Improvements**
  - Multiple commits indicate ongoing enhancement, likely focused on UI, data visualization, and integration with the API (see recent commit log).
  - Close coordination between API and Apps for improved workflows and data access.

11.3.3 calcofi4db: R Package & Data Management

11.3.3.1 R Package Initialization & Data Ingestion

- **New R Package: calcofi4db**
  - Initial commit and setup (commit), including functions for ingesting CSV datasets and metadata.
  - Refined change detection logic for source CSV files, improving tracking of table/field changes (commit).
  - Enhanced documentation and site via pkgdown.
  - Improved function naming and structure for ingestion (commits, commit).

11.3.4 calcofi4r: Spatial & Ecological Data Tools

11.3.4.1 Data Layers, Analysis, and Bug Fixes

- **Spatial Management Layers**
  - Ongoing integration of BOEM Wind Planning Areas, Marine Protected Areas, and SCCWRP management regions (issue, issue).
- **Analysis Functions**
  - Improved packages for ecological and spatial analysis, including new dependencies (commit).
- **User Feedback**
  - Addressing user-reported bugs such as deprecated function calls (issue).

11.3.5 Documentation (docs)

11.3.5.1 Infrastructure & Environment

- **Documentation Site Updates**
  - Added documentation for new packages and ingestion workflows (commit).
  - Improved environment handling for rendering with Quarto and Chromium (multiple commits Jan-Mar 2025).

- Updated diagrams and edge labels for database documentation.

11.3.6 Server

11.3.6.1 Backend Infrastructure

- **Backend Maintenance**
  - Numerous commits for improving server reliability, configuration, and deployment.
  - Indicates active backend support for API and Apps.

11.3.7 Workflows

11.3.7.1 Data Pipeline, Integration, and Registration

- **Workflow Automation**
  - Multiple commits show ongoing development of data ingestion, harmonization, and visualization workflows (commit, commit).
- **ODIS Registration**
  - Registering datasets with ODIS (using JSON-LD) for broader interoperability (issue).
- **Integration with External Data**
  - Ongoing work to load and harmonize diverse ecological datasets (bottle data, larvae, zooplankton, etc.).
- **Spatial Data Management**
  - Continued development of AOI (areas of interest), spatial buffer creation, and integration of management regions.

11.3.8 Key Themes & Impact

11.3.8.1 Integration & Interoperability

- Strong focus on connecting API, Apps, R packages, and backend infrastructure for seamless data access and visualization.
- Enhanced interoperability through ODIS registration and harmonized workflows.

11.3.8.2 Data Accessibility & Usability

- Improvements to API and Apps make ecological data more accessible to researchers and managers.
- Expanded support for spatial management areas and ecological datasets.

11.3.8.3 Infrastructure & Sustainability

- Investments in documentation, backend reliability, and workflow automation contribute to long-term sustainability and reproducibility.

11.3.9 For More Details

- Some results may be incomplete due to API limits.

- To view all commits/issues for 2025, visit each repository's GitHub UI and filter by year.

12 References

12.1 R packages

- API: plumber (Schloerke and Allen 2024)
- docs: Quarto (Allaire and Dervieux 2024)
- apps: Shiny (Chang et al. 2024)

Allaire, JJ, and Christophe Dervieux. 2024. *Quarto: R Interface to Quarto Markdown Publishing System*. https://github.com/quarto-dev/quarto-r.

Chang, Winston, Joe Cheng, JJ Allaire, et al. 2024. *Shiny: Web Application Framework for r*. https://shiny.posit.co/.

Schloerke, Barret, and Jeff Allen. 2024. *Plumber: An API Generator for r*. https://www.rplumber.io.